

# Module 2

## Routing and Addressing at Internet Scale

1 · Intra-AS routing protocols

2 · Inter-AS routing and AS policy

3 · Aggregation, NAT and IPv6

# A quick recap of routing protocols

We can categorise a routing protocol by the **algorithm** underneath it — in Module 1 we met **distance vector** and **link state**.

We can also categorise it by **where it runs** — inside one autonomous system, or between them.

| algorithm ↓<br>domain →                            | Intra-AS<br>within one autonomous system          | Inter-AS<br>between autonomous systems        |
|----------------------------------------------------|---------------------------------------------------|-----------------------------------------------|
| distance vector                                    | <b>RIP</b><br>Routing Information Protocol<br>now |                                               |
| link state                                         | <b>OSPF</b><br>Open Shortest Path First<br>now    |                                               |
| path vector<br>a close relative of distance vector |                                                   | <b>BGP</b><br>Border Gateway Protocol<br>next |



MODULE 2 · ROUTING INSIDE ONE ADMINISTRATIVE DOMAIN

# Intra-AS Routing Protocols

---

1 · Intra-AS routing protocols

2 · Inter-AS routing and AS policy

3 · Aggregation, NAT and IPv6

# An overview of intra-AS routing protocols

## RIP

Routing Information Protocol

**DISTANCE VECTOR**

this lecture

## OSPF

Open Shortest Path First

**LINK STATE**

this lecture

## IS-IS

Intermediate System to Intermediate System

**LINK STATE**

A link-state alternative to OSPF, widely used inside ISP networks. It works so much like OSPF that we will not cover it separately.

# A quick recap of the distance-vector routing protocol

- 1 Each router keeps a **distance vector** — its own best known cost to every destination, and the **next hop** on that path.  
↓
- 2 Each router sends its vector **only to its direct neighbours**, and receives theirs in return. No router ever learns the full topology.  
↓
- 3 On a neighbour's vector the router applies **relaxation** — if that neighbour offers a cheaper path, take it: update the cost and make that neighbour the next hop. Any entry that changed is advertised onward.

# A closer look at RIP

## 1 RIP is a distance-vector protocol

It runs the algorithm we have just recapped, within a single AS. But how does RIP measure the **cost** of each path?

## 2 Every link costs one hop

RIP measures the cost of a path by its number of **hops**. In the graph from Module 1, that is simply giving **every edge weight 1**.



distance from A to D = 3 hops

## 3 15 hops is as far as RIP counts

A path of more than **15 hops** is treated as unreachable — a metric of 16 means infinity.

**Why impose a limit at all?** To bound count to infinity — we come back to this shortly.

# A RIP router keeps two tables

Forwarding table at A

| destination     | next hop |
|-----------------|----------|
| 198.51.100.0/24 | B        |
| 203.0.113.0/24  | E        |
| 192.0.2.0/24    | B        |

## FORWARDING TABLE

You have seen this table since Module 1 — it is what A consults for **every datagram** it must forward.

But there is no **distance vector** in it. Where does RIP keep the one its algorithm works with?

THE TABLE THE DATAGRAMS USE, AND RIP'S OWN DATABASE

# A RIP router keeps two tables

RIP routing table at A

| destination     | metric | next hop | timer |
|-----------------|--------|----------|-------|
| 198.51.100.0/24 | 1      | B        | 180 s |
| 203.0.113.0/24  | 1      | E        | 180 s |
| 192.0.2.0/24    | 2      | B        | 180 s |

Forwarding table at A

| destination     | next hop |
|-----------------|----------|
| 198.51.100.0/24 | B        |
| 203.0.113.0/24  | E        |
| 192.0.2.0/24    | B        |

## RIP ROUTING TABLE

To run distance vector, every RIP router keeps a **second table** of its own — its **RIP routing table**.

For every destination it records more than the forwarding table does:

- the **metric** — how far away that destination is, its entry in the **distance vector**
- the **next hop** on that path
- a **timer** — RIP's own bookkeeping, giving the entry its time to live

RIP routing-table entries and timers: RFC 2453 §§3.5, 3.8

THE TABLE THE DATAGRAMS USE, AND RIP'S OWN DATABASE

## A RIP router keeps two tables

RIP routing table at A

| destination     | metric | next hop | timer |
|-----------------|--------|----------|-------|
| 198.51.100.0/24 | 1      | B        | 180 s |
| 203.0.113.0/24  | 1      | E        | 180 s |
| 192.0.2.0/24    | 2      | B        | 180 s |



Forwarding table at A

| destination     | next hop |
|-----------------|----------|
| 198.51.100.0/24 | B        |
| 203.0.113.0/24  | E        |
| 192.0.2.0/24    | B        |

### IT INSTALLS ROUTES

For each destination, RIP installs its chosen route — the destination and the next hop — into A's forwarding table.

THE TABLE THE DATAGRAMS USE, AND RIP'S OWN DATABASE

## A RIP router keeps two tables

RIP routing table at A

| destination     | metric | next hop | timer |
|-----------------|--------|----------|-------|
| 198.51.100.0/24 | 1      | B        | 180 s |
| 203.0.113.0/24  | 1      | E        | 180 s |
| 192.0.2.0/24    | 2      | B        | 180 s |



Forwarding table at A

| destination     | next hop |
|-----------------|----------|
| 198.51.100.0/24 | B        |
| 203.0.113.0/24  | E        |
| 192.0.2.0/24    | B        |

TWO TABLES, NOT ONE

They are kept separately, and may not always agree — RIP may hold on to an entry for its own **bookkeeping** after the forwarding table has dropped it. We will see an example of this shortly.

# The RIP conversation: Request and Response

Distance vector rests on routers **talking to their direct neighbours**. So how does RIP carry that conversation?



AFTER A REQUEST

## Requested

Return all routes or the selected routing-table entries requested by another router.

ROUGHLY EVERY 30 SECONDS

## Periodic

Send the current routing table to neighbouring RIP routers.

AFTER A ROUTE CHANGES

## Triggered

Send changed routing information shortly after the change occurs.

# An example of running RIP

RIP routing table at R1

| destination    | metric | next hop | timer |
|----------------|--------|----------|-------|
| 203.0.113.0/24 | 6      | R5       | 132 s |



## 1 · The route R1 has now

R1 reaches 203.0.113.0/24 through next hop **R5** at **cost metric 6**, with **132 s** left to live on the entry.

# An example of running RIP

RIP routing table at R1

| destination    | metric | next hop | timer |
|----------------|--------|----------|-------|
| 203.0.113.0/24 | 6      | R5       | 132 s |



## 2 · Periodic advertisement

Every RIP router periodically advertises its **distance vector** to its neighbours in a **Response**. Here R1 hears one such Response from R2.

# An example of running RIP

RIP routing table at R1

| destination    | metric | next hop | timer |
|----------------|--------|----------|-------|
| 203.0.113.0/24 | 6      | R5       | 132 s |



## 3 · What R2's vector says

R2 can reach 203.0.113.0/24 at **cost metric 2** — that is R2's own best known distance to it.

# An example of running RIP

RIP routing table at R1

| destination    | metric | next hop | timer |
|----------------|--------|----------|-------|
| 203.0.113.0/24 | 6      | R5       | 132 s |

VIA R2  $2 + 1 = 3 < \text{current } 6$



## 4 · Is it any better?

R1 is one hop further along, so going through R2 would cost  $2 + 1 = 3$ . That beats the current **6**, so R1 takes it.

# An example of running RIP

RIP routing table at R1

| destination    | metric | next hop | timer |
|----------------|--------|----------|-------|
| 203.0.113.0/24 | 3      | R2       | 180 s |

VIA R2  $2 + 1 = 3 < \text{current } 6$



## 5 · Replace the entry, then install it

- **Rewrite its RIP routing table.** Next hop becomes R2, cost metric becomes 3, and the 180-second timer starts again.
- **Install it.** R1 puts the new route into its forwarding table, and starts forwarding datagrams for this prefix to R2 instead of R5.

# An example of running RIP

RIP routing table at R1

| destination    | metric | next hop | timer   |
|----------------|--------|----------|---------|
| 203.0.113.0/24 | 3      | R2       | 180 s ↻ |



## 6 · R2 keeps it fresh

- Roughly every 30 seconds, R2 sends a **periodic Response** re-advertising the same route, telling R1 it is still current.
- Every refresh resets the **timer** back to 180 seconds.

# An example of running RIP

RIP routing table at R1

| destination    | metric | next hop | timer   |
|----------------|--------|----------|---------|
| 203.0.113.0/24 | 3      | R2       | expired |



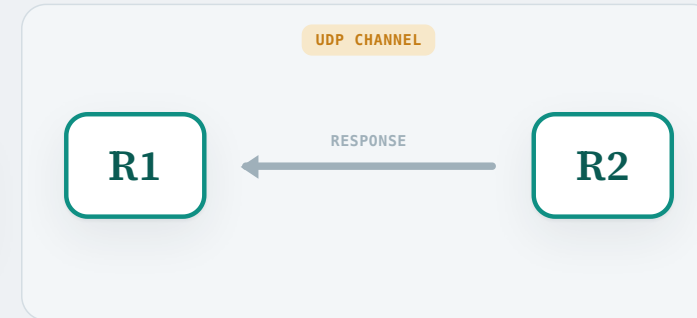
## 7 · What if R2 stops sending them?

- **The timer runs out.** No valid refresh arrives from R2, and after 180 seconds the entry expires.

# An example of running RIP

RIP routing table at R1

| destination    | metric | next hop | timer   |
|----------------|--------|----------|---------|
| 203.0.113.0/24 | 16     | R2       | expired |



## 7 · What if R2 stops sending them?

- **The timer runs out.** No valid refresh arrives from R2, and after 180 seconds the entry expires.
- **Mark it unreachable.** R1 sets the cost metric to **16** — RIP's infinity — and immediately drops the route from its **forwarding table**.
- **Keep it, to tell the neighbours.** The entry stays in the RIP routing table for another **120 seconds**, advertised with metric 16 so the neighbours learn the destination is gone.
- **Then remove it.** After those 120 seconds, R1 deletes the entry from its routing table altogether.

# Why a distance vector can count to infinity

A distance-vector protocol suffers from the **count-to-infinity problem**.

The cause is what a distance vector carries: **distances, not paths**. A neighbour is told *how far*, never *through whom* — so a router cannot see that the route it is offered runs back through itself.

Suppose the link onward to **R3** fails. R2 hears that R1 can still reach R3, while R1's own route to R3 runs through R2 — so each forwards to the other, and their costs climb together, one hop at a time.



# How RIP limits count to infinity

## POISON REVERSE

### Send it back as infinity

If R1's route to a destination runs through R2, R1 advertises that destination back to R2 at a cost of **infinity** — so R2 never tries to reach it through R1.

Only a **heuristic**. It breaks a loop between **two** routers, but not one running through three or more.

## COUNT ONLY TO 16

### Make infinity deliberately small

So the counting cannot run away for long: at **16** a route is already infinity. That is why the maximum path length in RIP is **15 hops**.

This is RIP's **fundamental solution** to the count-to-infinity problem.

# OSPF · Open Shortest Path First

**What it is.** OSPF is an intra-AS routing protocol which uses the **link-state** routing algorithm.

**Protocol channel.** OSPF carries its messages **directly over IP**, without TCP or UDP.

**What they carry.** Those messages carry the **link-state advertisements** — so before going further, a reminder of how link state works.

# A quick recap of the link-state routing protocol

- 1 Each router creates an **advertisement** for its local information, and tells the others.
- ↓
- 2 Every router uses all the advertisements to build the **same topology**.
- ↓
- 3 Each router runs a **shortest-path algorithm** over that topology to derive its forwarding table.

# OSPF provides useful features for a large AS

## Authentication

Routers accept updates only from configured participants. Without it, any host on the segment could advertise a **cost-1 path** to a prefix and pull that traffic through itself — to drop it, or to read it.

## Operator-defined costs

The operator sets each link's cost, and so decides which paths OSPF prefers. Costs can follow **bandwidth**, so a fast link beats a slow one — a distinction hop count cannot make — or a metered link can be priced high to keep it as a backup.

## Area routing

A large AS can be divided into **areas**, so that OSPF scales: flooding and the shortest-path computation stay **inside** an area, and only summaries cross between them.

# Why can a large AS strain OSPF?

## One large AS

many routers · one shared OSPF topology



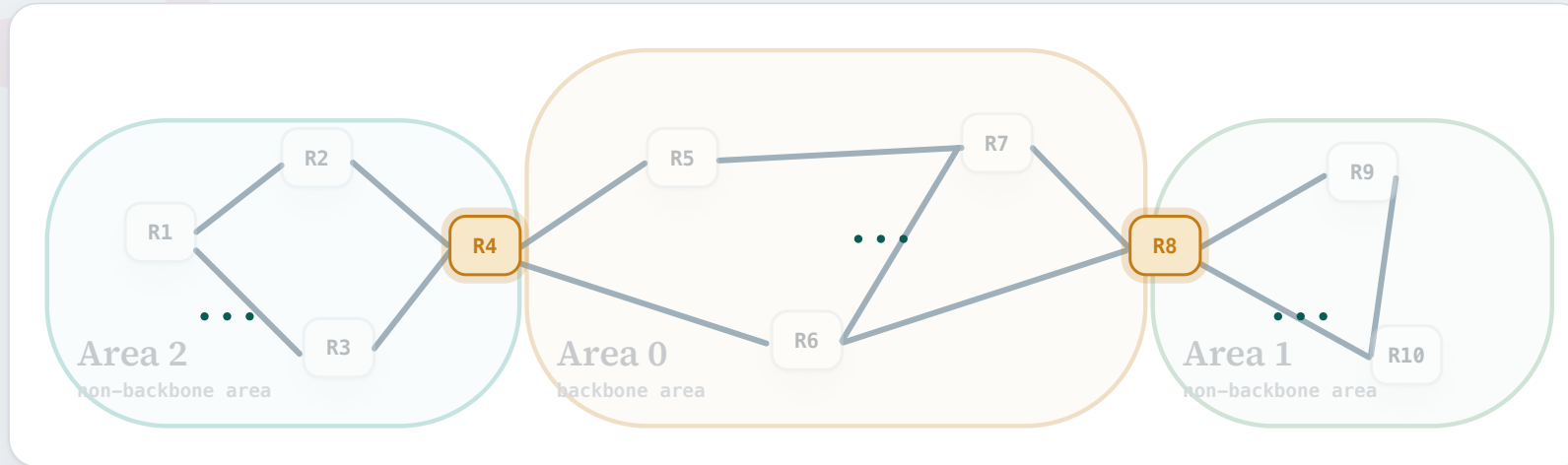
### More memory

Every router stores the complete AS topology in its link-state database.

### Slower convergence

Every change must be flooded widely before all routers calculate the new topology.

# OSPF divides one AS into smaller areas



## Non-backbone areas

Areas 1 and 2 each contain a smaller group of routers and networks.

## Backbone area

Area 0 connects the non-backbone areas and carries inter-area reachability.

## Area border routers

R4 and R8 each participate in Area 0 and one non-backbone area.

# How does hierarchical OSPF fit together?



## Inside an area

- A router does its link-state flooding only within its own area, and stores that area's topology.
- Each router then runs the shortest-path algorithm over that topology to reach destinations inside its area.

## Between areas

- ABRs carry condensed inter-area reachability through the backbone, Area 0.
- For example, a route from Area 2 to Area 1 can follow:  
**Area 2 → R4 → Area 0 → R8 → Area 1**

OSPF areas and inter-area routing: RFC 2328 §§3, 3.1–3.3, 4.1

# Intra-AS routing recap

## ● RIP is distance vector.

- It measures distance in router hops, and caps a path at 15.
- Neighbours exchange distance vectors, kept alive by periodic Responses.

## ● OSPF is link state.

- Routers flood advertisements, so each builds the same topology.
- A large AS is divided into areas so that OSPF scales.



MODULE 2 · ROUTING BETWEEN ADMINISTRATIVE DOMAINS

# Inter-AS Routing and AS Policy

---

1 · Intra-AS routing protocols

**2 · Inter-AS routing and AS policy**

3 · Aggregation, NAT and IPv6

# The Internet is a network of autonomous systems



SO FAR, INSIDE ONE AS

- RIP and OSPF both run **within a single AS** — many routers forming **several subnets**, all managed by one organisation.
- Each subnet is identified by a **network prefix**, written in **CIDR** notation — AS A holds 192.0.2.0/25 and 192.0.2.128/25.
- So 192.0.2.1 sending to 192.0.2.200 stays **inside AS A**: that is **intra-AS routing**.

# The Internet is a network of autonomous systems



## INTER-AS ROUTING PROBLEM

Now 192.0.2.1 sends instead to 203.0.113.42 — an address in **AS D**.

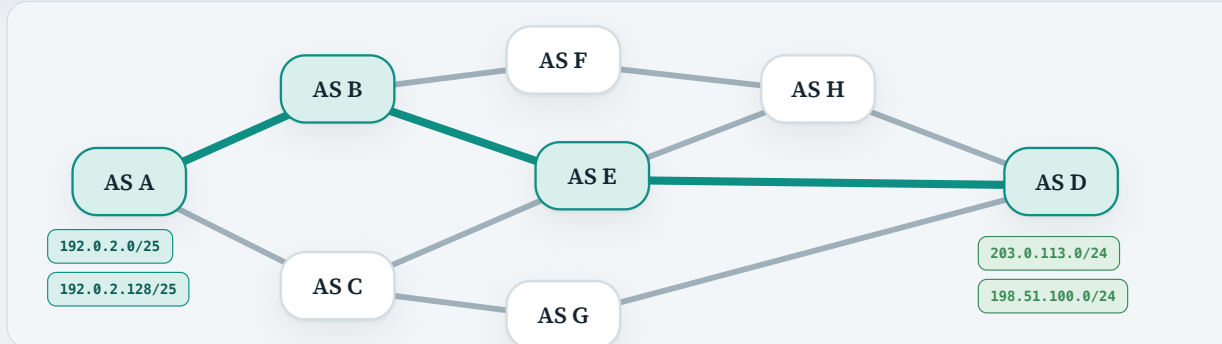
# BGP is the Internet's inter-AS routing protocol

BGP is the Internet's answer to that problem. Here is how it does it.

## LEARN EXTERNAL REACHABILITY

- Route-finding over a graph of **nodes** and **edges**, just as in Module 1 — except a node is no longer a router but a whole **autonomous system**, holding one or more subnets.
- Unlike Module 1, a destination is not a single IP address. It is a **destination prefix** in CIDR notation, like `203.0.113.0/24`, identifying a subnet inside some AS.
- For every **source–destination pair**, find a path across that graph — a path made of several ASes.

e.g. a path from AS A to AS D: **AS A → AS B → AS E → AS D**



BGP purpose and operation: RFC 4271 §§1, 3, 9.1

# BGP is the Internet's inter-AS routing protocol

BGP is the Internet's answer to that problem. Here is how it does it.

## LEARN EXTERNAL REACHABILITY

- Route-finding over a graph of **nodes** and **edges**, just as in Module 1 — except a node is no longer a router but a whole **autonomous system**, holding one or more subnets.
- Unlike Module 1, a destination is not a single IP address. It is a **destination prefix** in CIDR notation, like `203.0.113.0/24`, identifying a subnet inside some AS.
- For every **source–destination pair**, find a path across that graph — a path made of several ASes.

e.g. a path from AS A to AS D: **AS A → AS B → AS E → AS D**

## DISTRIBUTE IT INSIDE THE AS

Once those external paths are known, the AS's own routers must be told, so they know how to forward packets leaving the AS.

## SELECT A PREFERRED ROUTE

Again several paths may reach the same destination. In Module 1 the criterion was simple — the **shortest** path, one least cost. Here it is **multidimensional**: economic incentives, delay, and more. BGP supplies the criteria that pick the best one.

WHO PARTICIPATES IN BGP?

# BGP is run by BGP-speaking routers

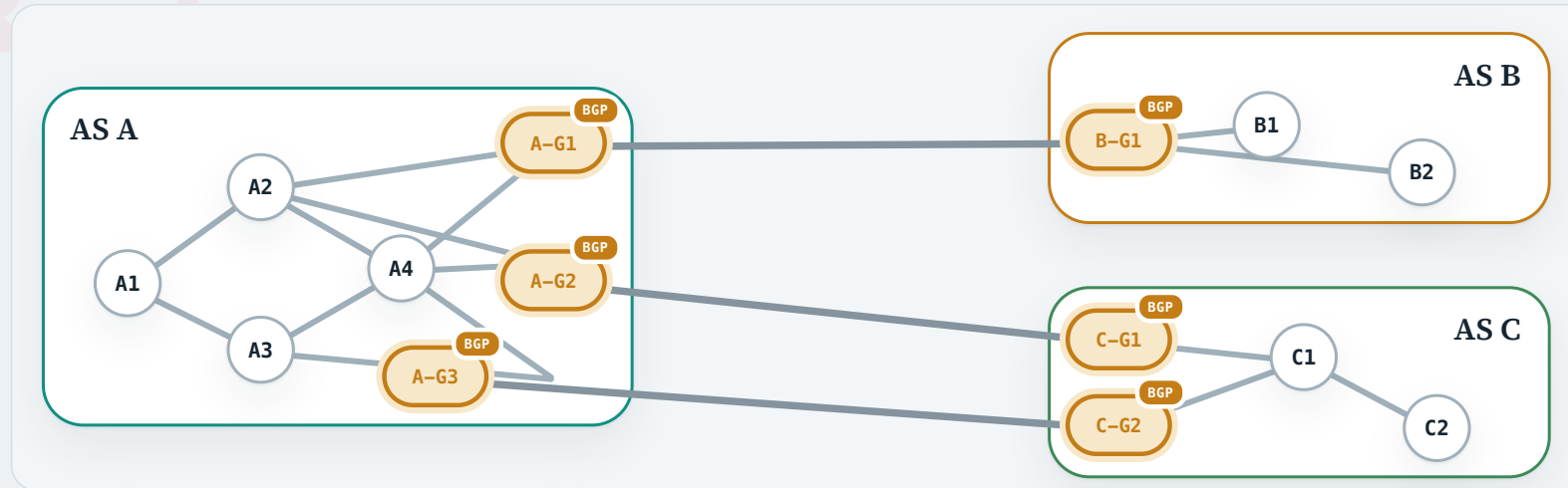


## BGP SPEAKER

A router that runs BGP and maintains configured BGP sessions with selected peer routers. Each AS decides which routers will participate.

WHO PARTICIPATES IN BGP?

# BGP is run by BGP-speaking routers



USUALLY AT THE EDGE

A border or gateway router links directly to a router in another AS. Such routers normally run BGP; here, all six gateways are BGP speakers.

WHO PARTICIPATES IN BGP?

# BGP is run by BGP-speaking routers



NOT ONLY GATEWAYS

BGP is not limited to gateways: internal routers such as A2, A4, B1, and C1 can also be BGP speakers.

WHO PARTICIPATES IN BGP?

# BGP is run by BGP-speaking routers



**NOT EVERY NODE**

Only selected routers run BGP; ordinary hosts and many internal routers do not participate.

# A BGP session is external or internal



## External BGP (eBGP)

| this is how an AS **learns external reachability**

Sessions between BGP speakers in **different ASes**, usually the border speakers each side of the boundary. Using **eBGP**, an AS learns **AS-level paths** to destinations outside itself.

## Internal BGP (iBGP)

| this is how the AS **distributes it inside**

Sessions between BGP speakers in the **same AS**. Using **iBGP**, all the routers inside the AS come to know those external paths, so each of them can forward towards the outside.

A STRUCTURED CONVERSATION BETWEEN TWO BGP SPEAKERS

# A BGP session is a structured conversation



## TCP · THE CHANNEL

Both iBGP and eBGP use TCP as the channel for the conversation. We will study TCP later in this course.

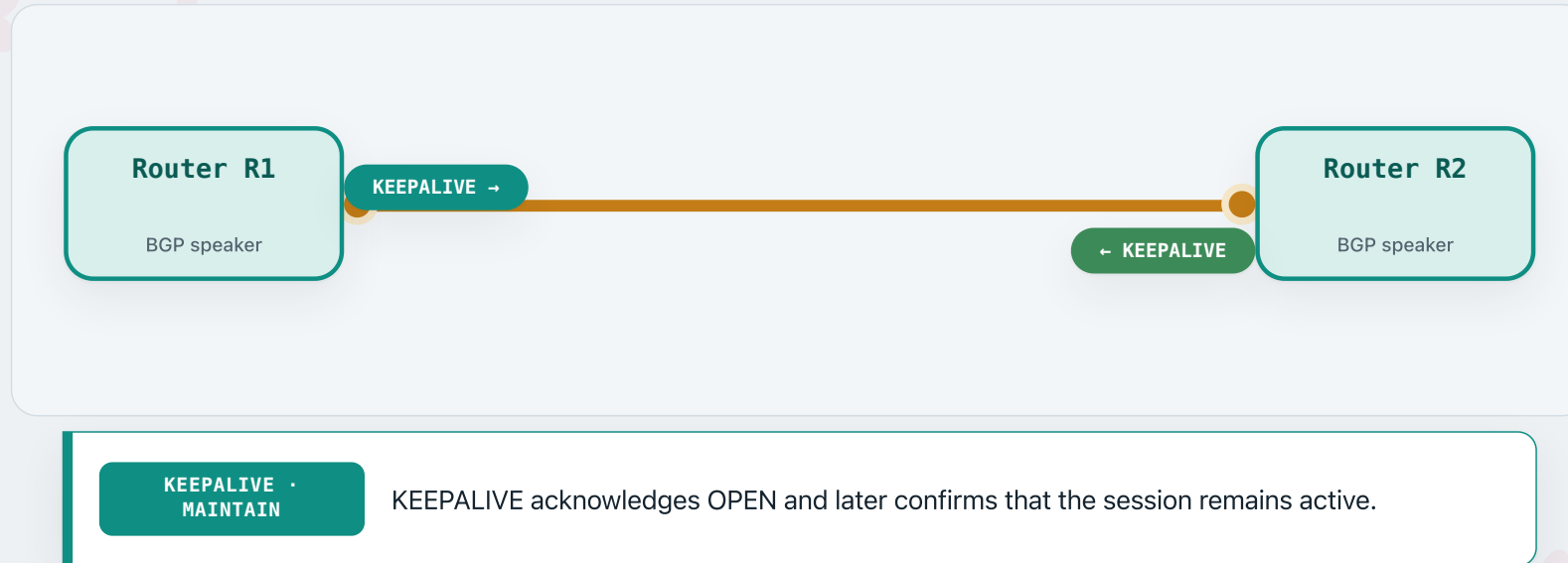
A STRUCTURED CONVERSATION BETWEEN TWO BGP SPEAKERS

# A BGP session is a structured conversation

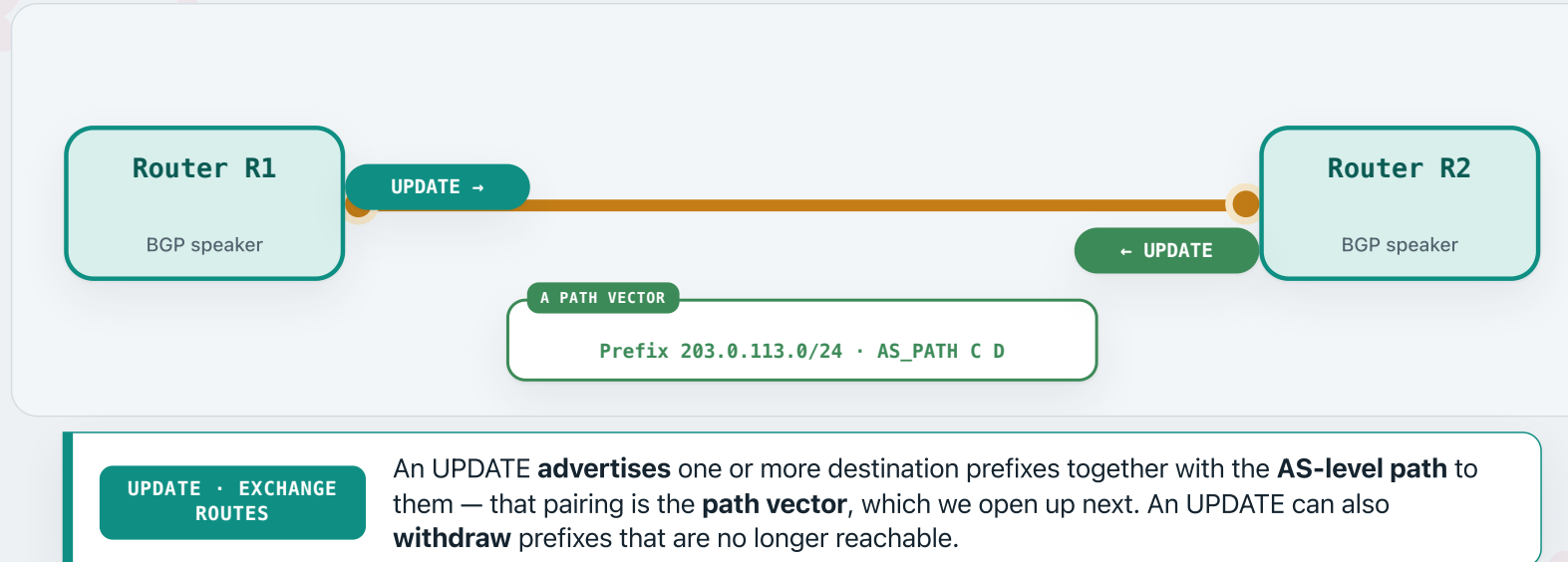


A STRUCTURED CONVERSATION BETWEEN TWO BGP SPEAKERS

# A BGP session is a structured conversation



# A BGP session is a structured conversation



## A closer look at a BGP path vector

Unlike a distance-vector protocol, what BGP advertises to a neighbour is a path vector.

### ONE PATH-VECTOR ROUTE

|          |                    |
|----------|--------------------|
| NLRI     | 203.0.113.0/24     |
| AS_PATH  | AS E · AS H · AS D |
| NEXT_HOP | E1                 |

#### NLRI

**Network Layer Reachability Information:** the destination prefix being advertised.

#### AS\_PATH

The complete **AS-level path leading to that destination prefix**.

#### NEXT\_HOP

The IP address of the next router to which traffic should be forwarded towards that prefix.

## Watch one BGP route travel across three ASes



1 · AS D OWNS THE  
PREFIX

203.0.113.0/24 is one of the subnets inside AS D. For hosts elsewhere on the Internet to reach it, AS D has to **advertise this destination prefix to the other ASes.**

BGP UPDATE, path attributes, and import/export policy: RFC 4271 §§3, 4.3, 5.1.2–5.1.3, 9.1–9.2

## Watch one BGP route travel across three ASes



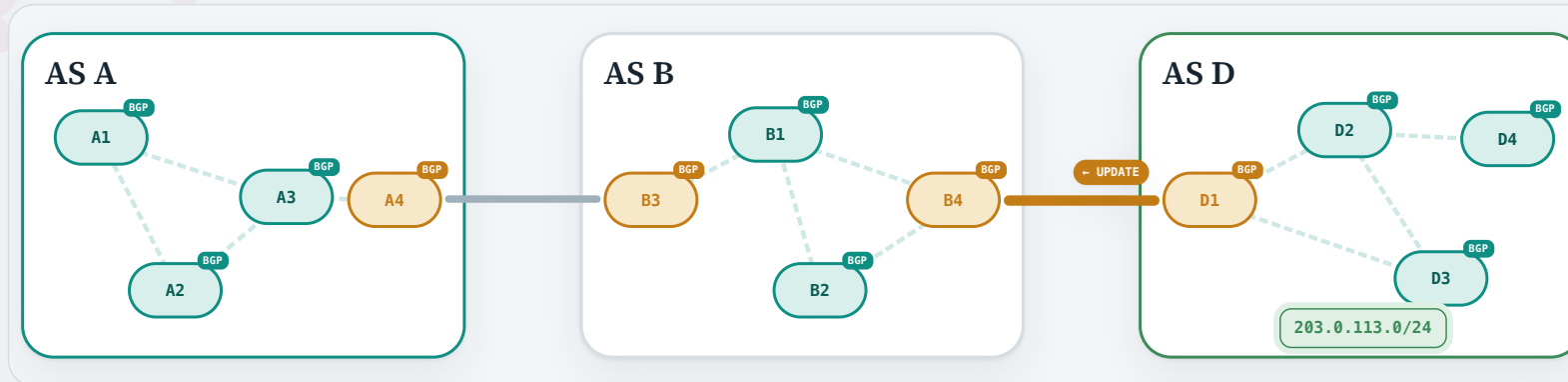
### 2 · AS D EXPORT POLICY

AS D has a neighbour it could advertise to — AS B. But before it does, it consults its **export policy**: is advertising this prefix to this neighbour something AS D is willing to do?

*We will return to export policy later.*

BGP UPDATE, path attributes, and import/export policy: RFC 4271 §§3, 4.3, 5.1.2–5.1.3, 9.1–9.2

## Watch one BGP route travel across three ASes



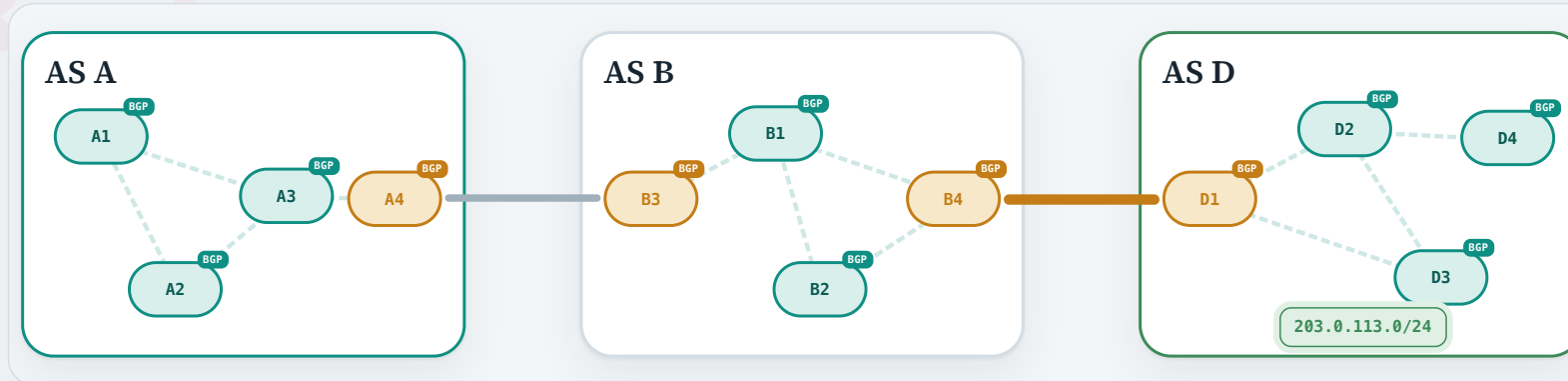
3 · EBGP: D → B

Suppose AS D permits the export. D1 sends an UPDATE to B4 over their established eBGP session.

```
NLRI      203.0.113.0/24
AS_PATH   AS D
NEXT_HOP  D1
```

BGP UPDATE, path attributes, and import/export policy: RFC 4271 §§3, 4.3, 5.1.2–5.1.3, 9.1–9.2

## Watch one BGP route travel across three ASes



### 4 · AS B IMPORT POLICY

AS B applies its **import policy**. B4 may accept or reject D1's advertisement.  
*We will return to import policy later.*

BGP UPDATE, path attributes, and import/export policy: RFC 4271 §§3, 4.3, 5.1.2–5.1.3, 9.1–9.2

# Watch one BGP route travel across three ASes



## 5 · IBGP INSIDE B

Suppose AS B accepts. B4 then shares the accepted route using **internal BGP (iBGP)** with AS B's other BGP speakers; in particular, B3 learns it.

|          |                  |
|----------|------------------|
| NLRI     | 203.0.113.0/24   |
| AS_PATH  | AS D · unchanged |
| NEXT_HOP | D1               |

BGP UPDATE, path attributes, and import/export policy: RFC 4271 §§3, 4.3, 5.1.2–5.1.3, 9.1–9.2

## Watch one BGP route travel across three ASes



### 6 · AS B EXPORT POLICY

AS B now knows it can reach the prefix, and it would like its own neighbours to know that too. It has a neighbour it could advertise to — AS A. But first it consults its **export policy**: is carrying other ASes' traffic to this prefix something AS B is willing to do?

BGP UPDATE, path attributes, and import/export policy: RFC 4271 §§3, 4.3, 5.1.2–5.1.3, 9.1–9.2

# Watch one BGP route travel across three ASes



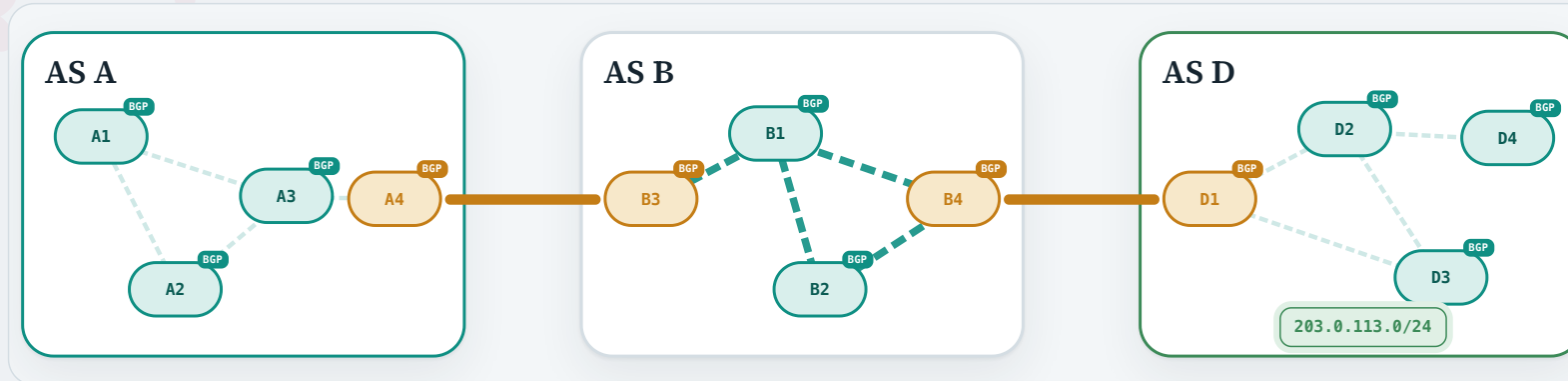
7 · EBGP: B → A

Suppose AS B permits the export. B3 **prepends AS B** onto the `AS_PATH` and sets `NEXT_HOP` to its own IP address, then sends this path vector in an UPDATE to AS A's border router A4.

```
NLRI      203.0.113.0/24
AS_PATH   AS B · AS D
NEXT_HOP  B3
```

BGP UPDATE, path attributes, and import/export policy: RFC 4271 §§3, 4.3, 5.1.2–5.1.3, 9.1–9.2

## Watch one BGP route travel across three ASes



### 8 · AS A IMPORT POLICY

AS A applies its **import policy**. A4 may accept or reject B3's advertisement.

BGP UPDATE, path attributes, and import/export policy: RFC 4271 §§3, 4.3, 5.1.2–5.1.3, 9.1–9.2

## Watch one BGP route travel across three ASes



9 · IBGP INSIDE A

Suppose AS A accepts. A4 then shares the accepted route using **internal BGP (iBGP)** with AS A's other BGP speakers.

|          |                |
|----------|----------------|
| NLRI     | 203.0.113.0/24 |
| AS_PATH  | AS B · AS D    |
| NEXT_HOP | B3             |

BGP UPDATE, path attributes, and import/export policy: RFC 4271 §§3, 4.3, 5.1.2–5.1.3, 9.1–9.2

## Watch one BGP route travel across three ASes

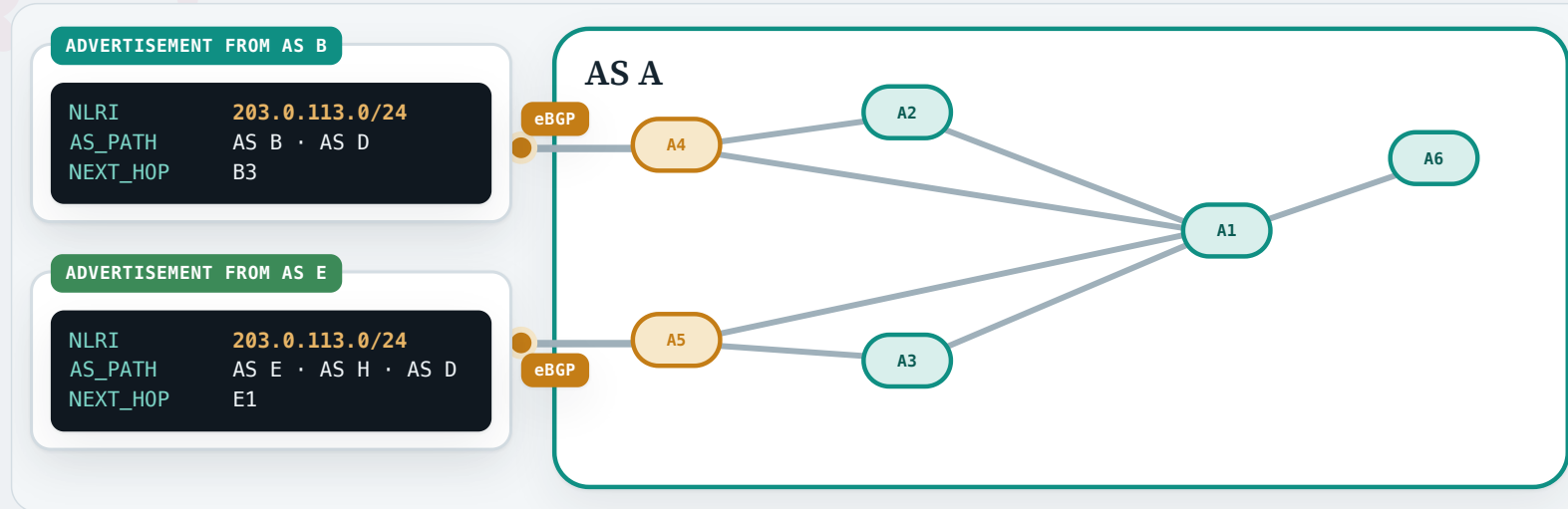


10 · EVERY ROUTER IN  
AS A KNOWS THE WAY

Every BGP speaker in AS A now knows that 203.0.113.0/24 is reachable **via AS B, then AS D** — carried between the ASes by **eBGP** and spread inside each one by **iBGP**.

BGP UPDATE, path attributes, and import/export policy: RFC 4271 §§3, 4.3, 5.1.2–5.1.3, 9.1–9.2

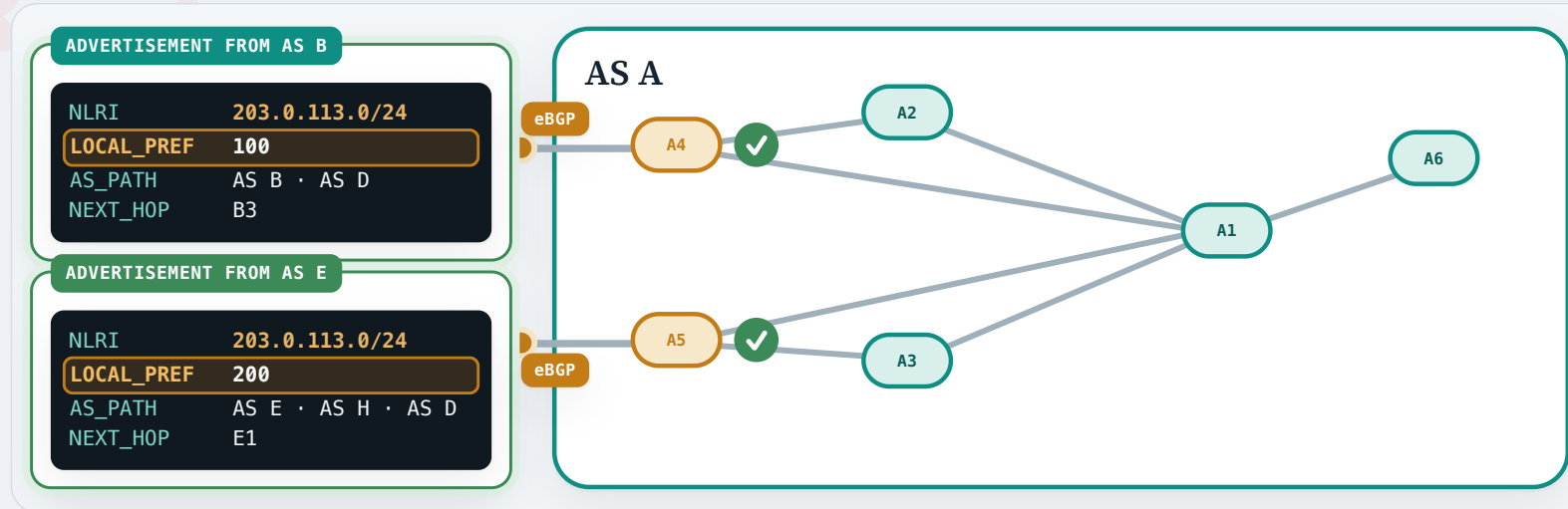
# eBGP import policy: which routes an AS prefers



## 1 · ONE PREFIX, TWO ROUTES

Two advertisements for the same prefix, 203.0.113.0/24, arrive over separate eBGP sessions at two different AS A border gateways. **Which one should AS A allow to import?**

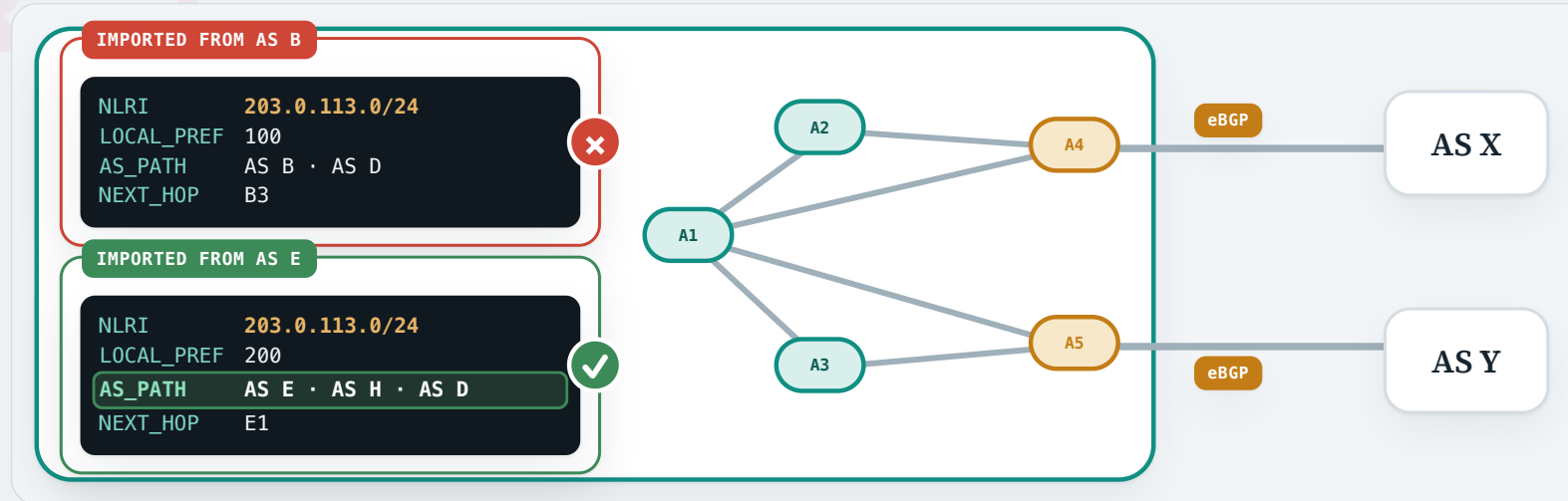
# eBGP import policy: which routes an AS prefers



2 · ACCEPT BOTH, AND RANK THEM

AS A's import policy accepts both routes and attaches a `LOCAL_PREF` to each — 100 and 200 — for use inside AS A. The higher value is the one AS A prefers.

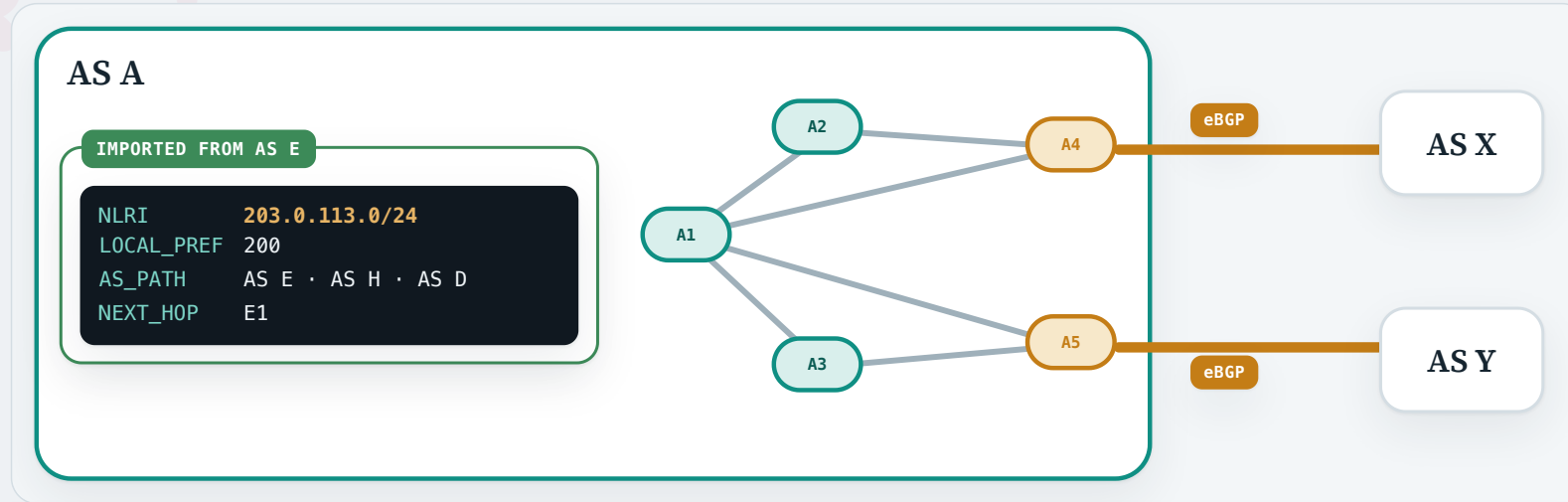
# eBGP export policy: what an AS advertises



## 1 · SELECT AN AS PATH

Both routes were imported, but the one through AS E carries the higher LOCAL\_PREF, so AS A selects **AS E · AS H · AS D** for 203.0.113.0/24.

# eBGP export policy: what an AS advertises

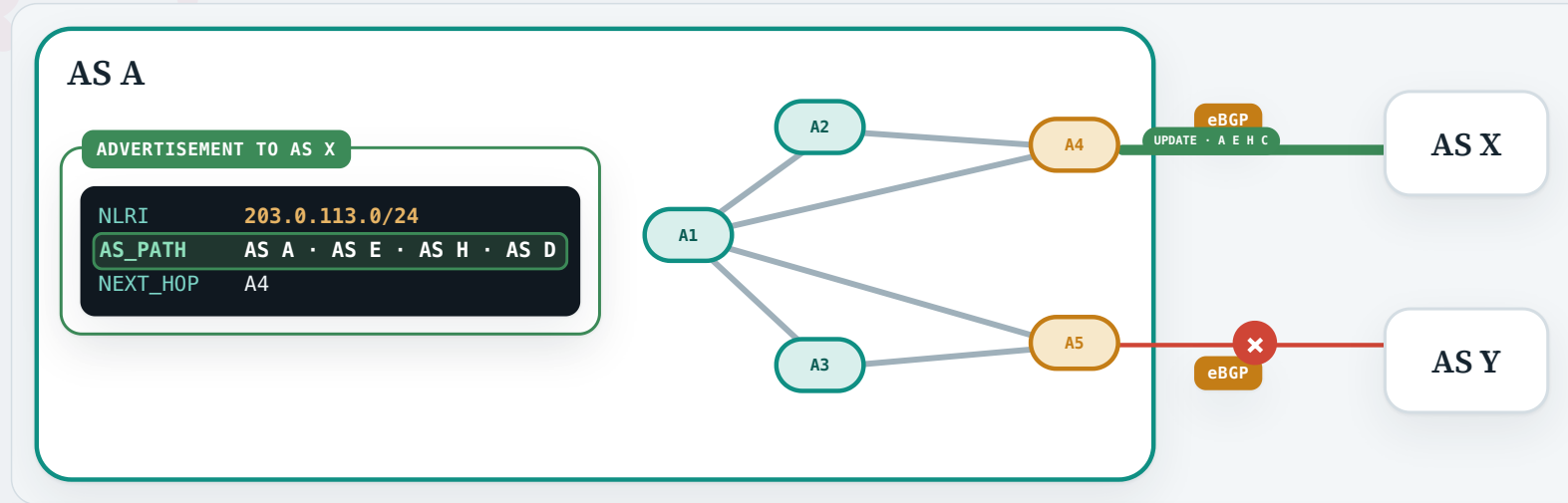


## 2 · ADVERTISE IT ONWARD?

AS A can now reach the prefix, and it has two neighbours it could tell — AS X and AS Y. **Which of them should AS A advertise this route to?**

Per-neighbour export policy and route dissemination: RFC 4271 §§9.2, 5.1.5; RFC 8212

# eBGP export policy: what an AS advertises



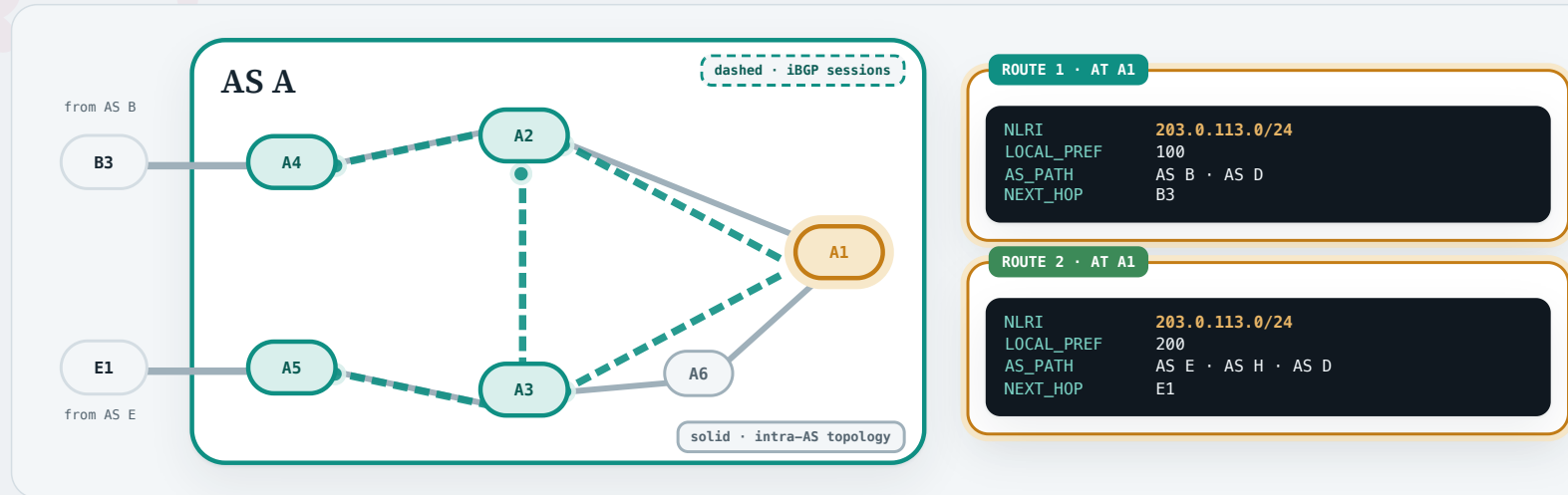
## 3 · EXPORT POLICY DECIDES

AS A's export policy answers per neighbour: it prepends itself and advertises **AS A · AS E · AS H · AS D** to AS X, while withholding it from AS Y.

Per-neighbour export policy and route dissemination: RFC 4271 §§9.2, 5.1.5; RFC 8212

ONE NEXT HOP PER PREFIX · LOCAL\_PREF → AS\_PATH → INTRA-AS COST

# How each BGP router builds its forwarding table



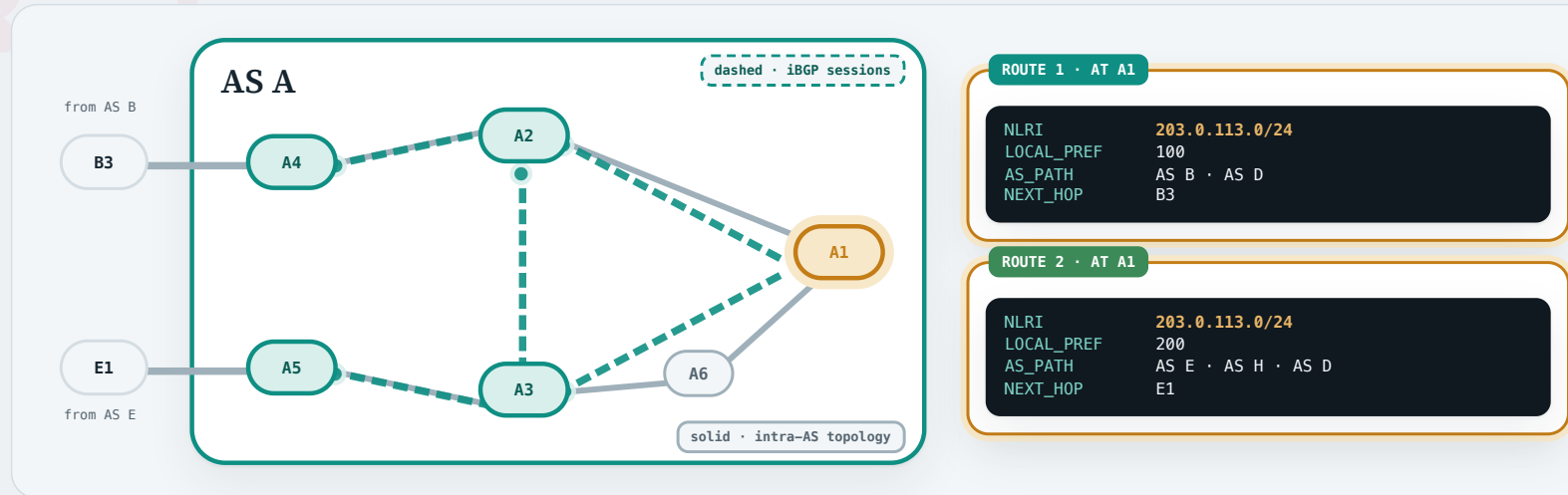
## THE SETUP

Over iBGP, router A1 has learned **two routes** to 203.0.113.0/24 — one from A4, one from A5.

iBGP, distributed selection, LOCAL\_PREF, and intra-AS cost to NEXT\_HOP: RFC 4271 §§3, 5.1.5, 9.1.1–9.1.2

ONE NEXT HOP PER PREFIX · LOCAL\_PREF → AS\_PATH → INTRA-AS COST

# How each BGP router builds its forwarding table



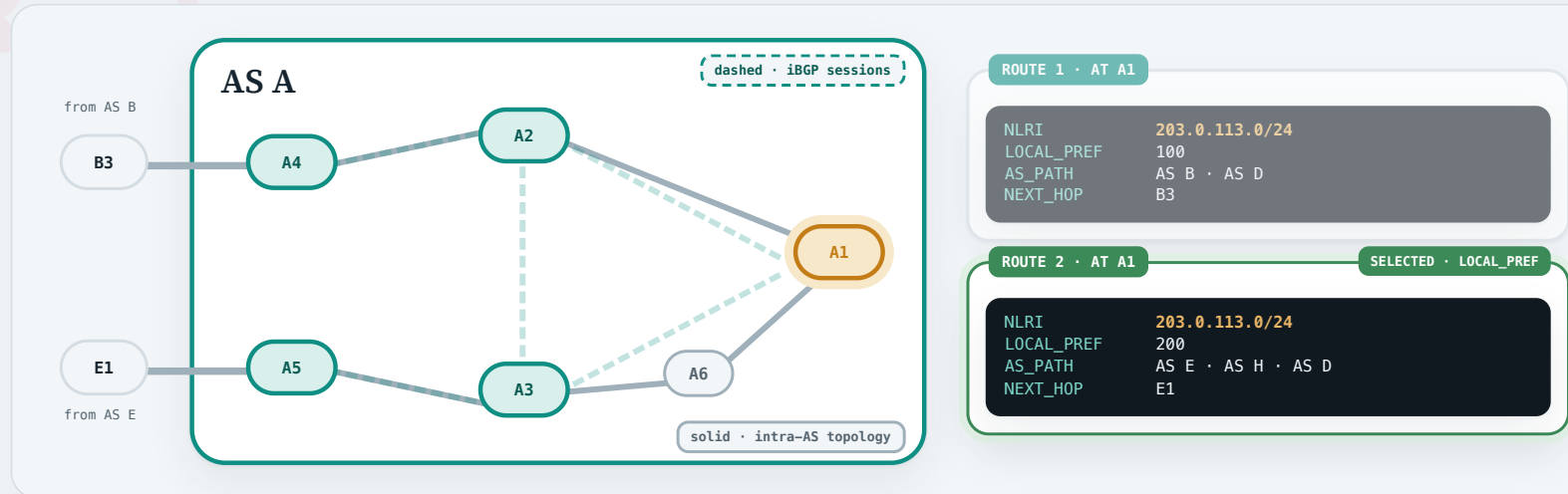
## THE QUESTION

A1 must put **one** next hop for this prefix into its forwarding table. **Which of the two routes should it pick?**

iBGP, distributed selection, LOCAL\_PREF, and intra-AS cost to NEXT\_HOP: RFC 4271 §§3, 5.1.5, 9.1.1–9.1.2

ONE NEXT HOP PER PREFIX · LOCAL\_PREF → AS\_PATH → INTRA-AS COST

# How each BGP router builds its forwarding table



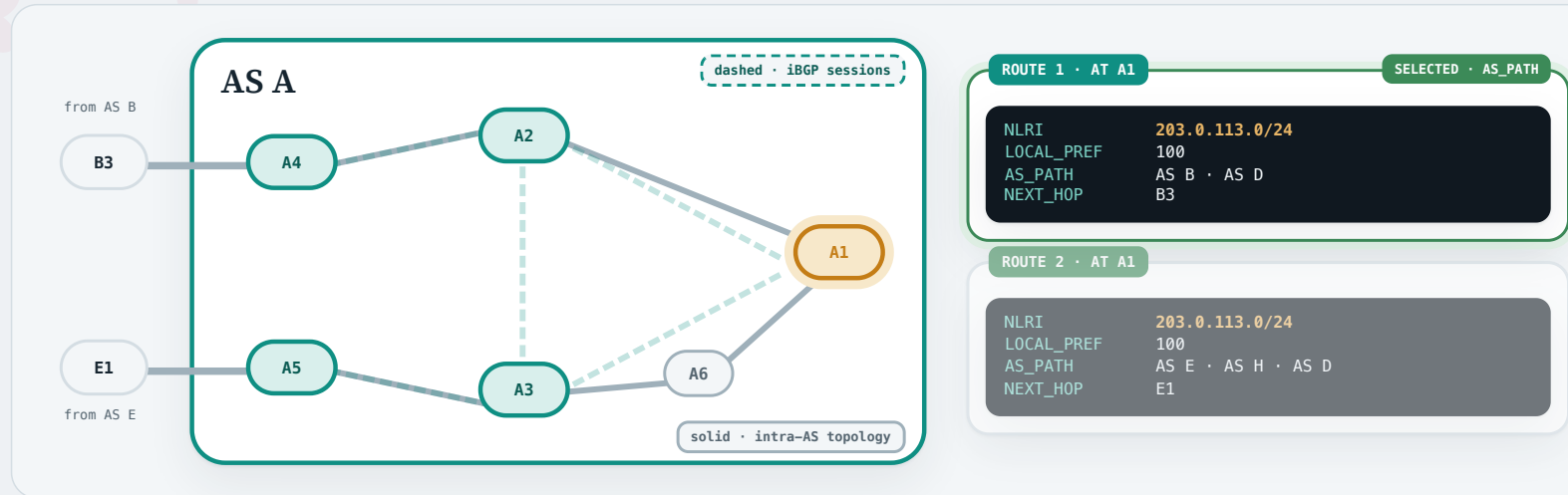
## 1 · AS-WIDE POLICY FIRST

The first tie-breaker is `LOCAL_PREF`, the AS-wide preference from import policy: 200 beats 100, so **Route 2 wins**.

iBGP, distributed selection, `LOCAL_PREF`, and intra-AS cost to `NEXT_HOP`: RFC 4271 §§3, 5.1.5, 9.1.1–9.1.2

ONE NEXT HOP PER PREFIX · LOCAL\_PREF → AS\_PATH → INTRA-AS COST

# How each BGP router builds its forwarding table



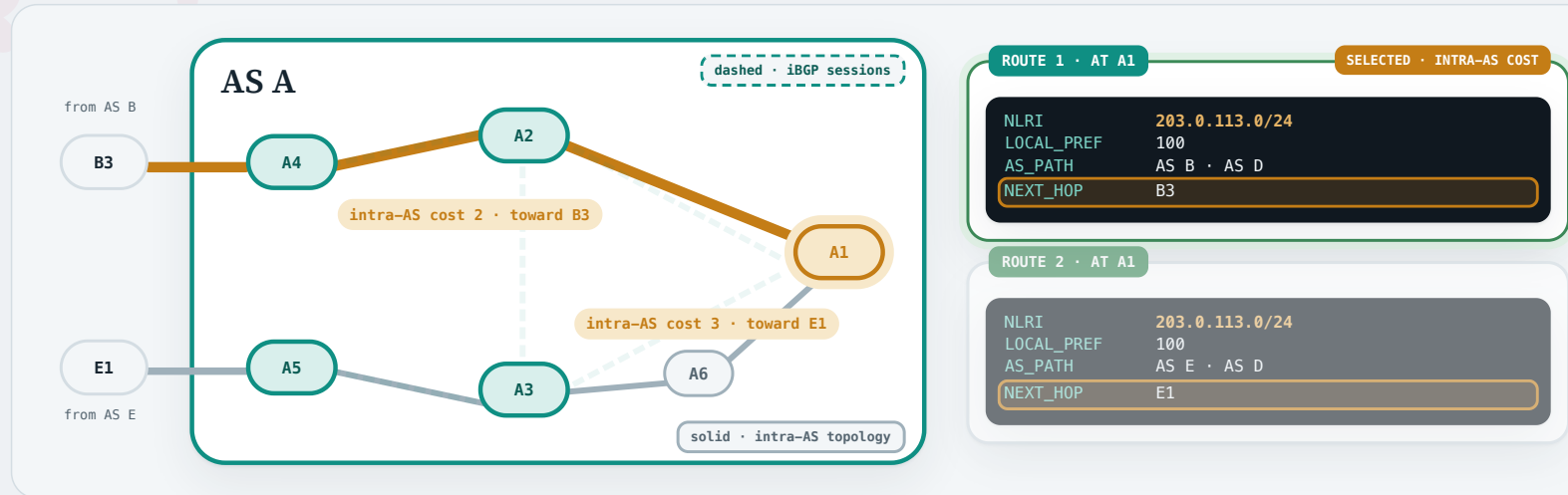
2 · THEN AS\_PATH LENGTH

If LOCAL\_PREF ties, A1 compares AS-level path lengths and takes the shorter: **Route 1 wins** with two ASes rather than three.

iBGP, distributed selection, LOCAL\_PREF, and intra-AS cost to NEXT\_HOP: RFC 4271 §§3, 5.1.5, 9.1.1–9.1.2

ONE NEXT HOP PER PREFIX · LOCAL\_PREF → AS\_PATH → INTRA-AS COST

# How each BGP router builds its forwarding table

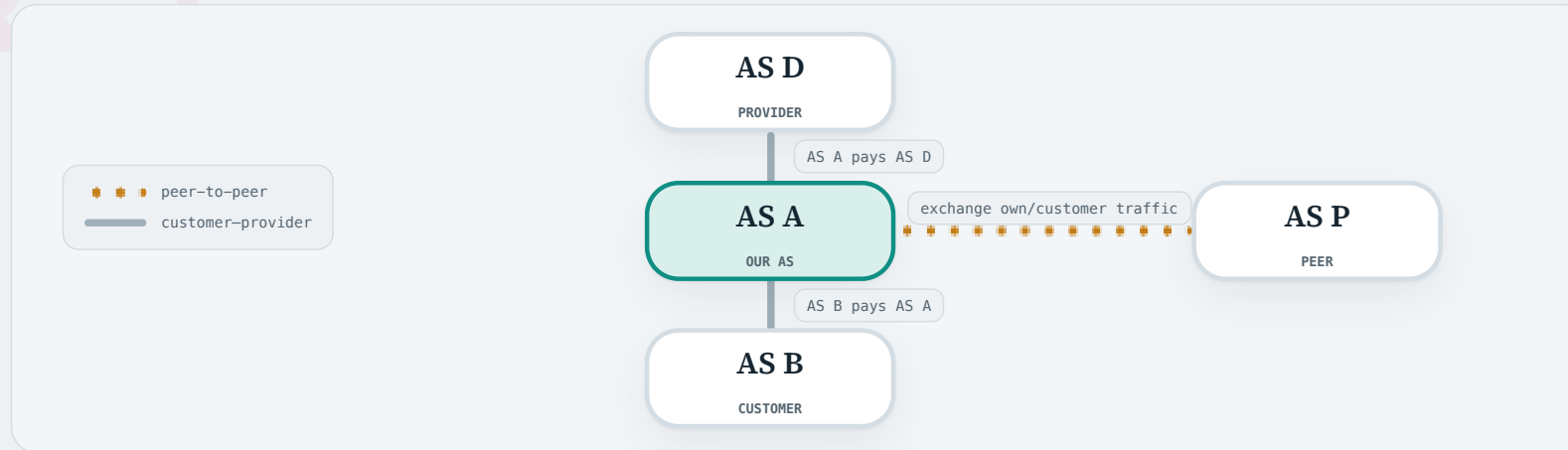


## 3 · THEN HOT-POTATO ROUTING

If both BGP attributes tie, A1 turns to AS A's **own internal topology** and asks its own intra-AS routing protocol which next hop is closer: B3 at cost 2 beats E1 at cost 3, so **Route 1 wins — hot-potato routing**.

iBGP, distributed selection, LOCAL\_PREF, and intra-AS cost to NEXT\_HOP: RFC 4271 §§3, 5.1.5, 9.1.1–9.1.2

# Business relationships between ASes

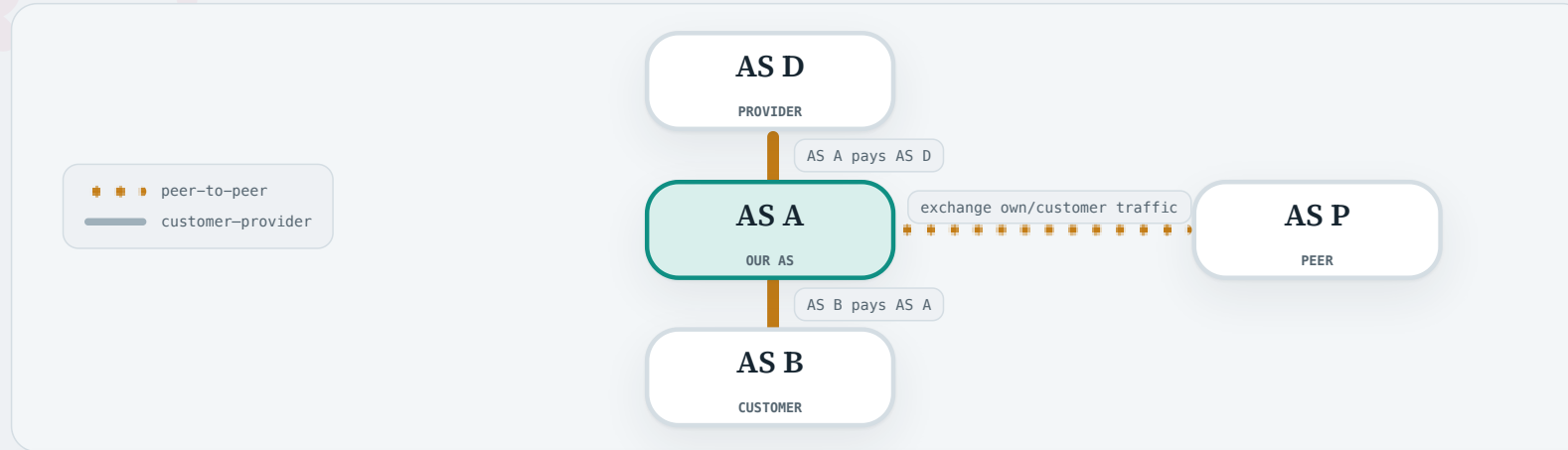


## THE QUESTION

**What drives an AS's import and export policy?**

The answer lies in the **business relationships** between ASes — who pays whom for carrying traffic.

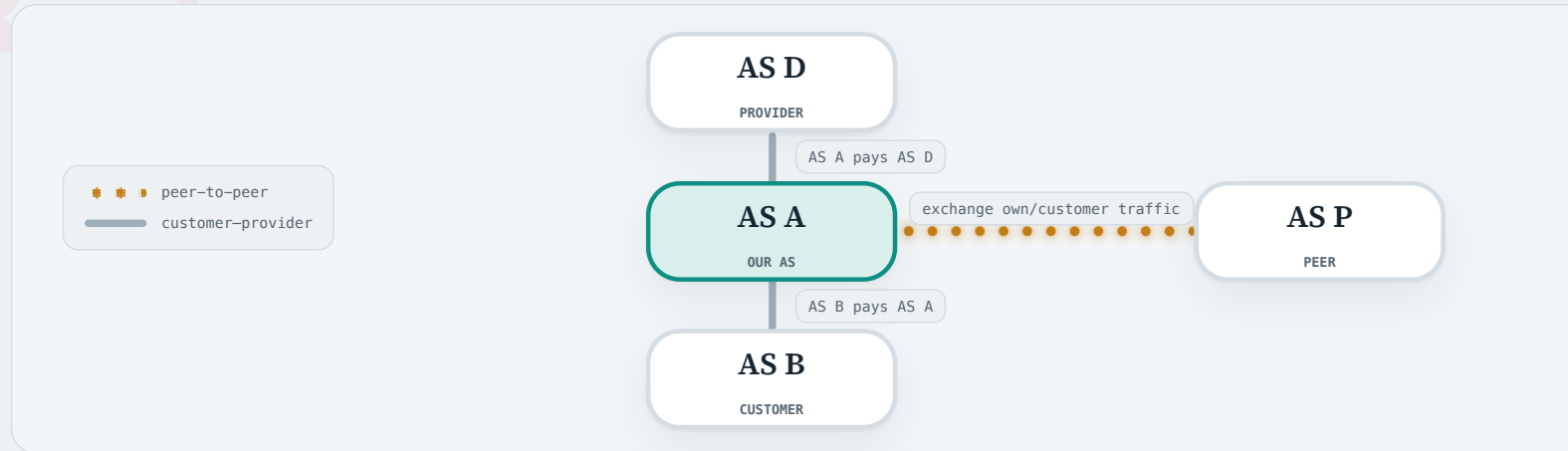
# Business relationships between ASes



## CUSTOMER → PROVIDER

The customer pays the provider for transit to the wider Internet.  
e.g. AS A buys its connectivity to the rest of the Internet from AS D, and pays AS D for it.

# Business relationships between ASes

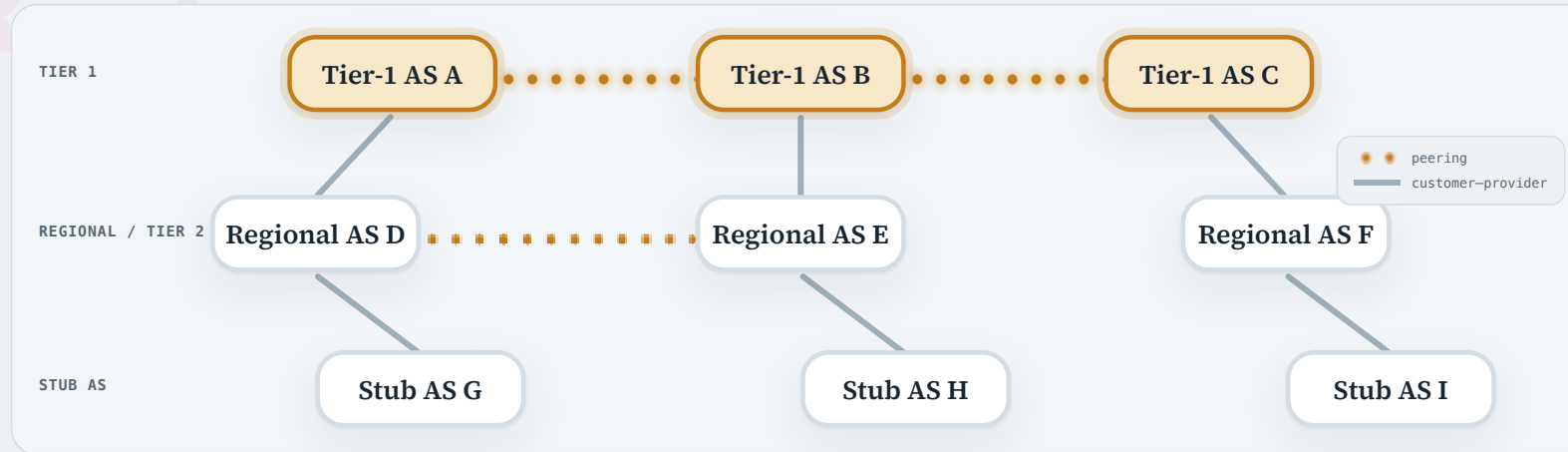


## PEER ↔ PEER

Peers commonly exchange traffic for themselves and their customers—not general transit for one another.  
e.g. AS A and AS P exchange roughly as much traffic as each other, so neither pays the other and neither depends on the other for connectivity.

HOW CUSTOMER, PROVIDER, AND PEER RELATIONSHIPS REPEAT ACROSS THE INTERNET

## These relationships create a commercial AS hierarchy



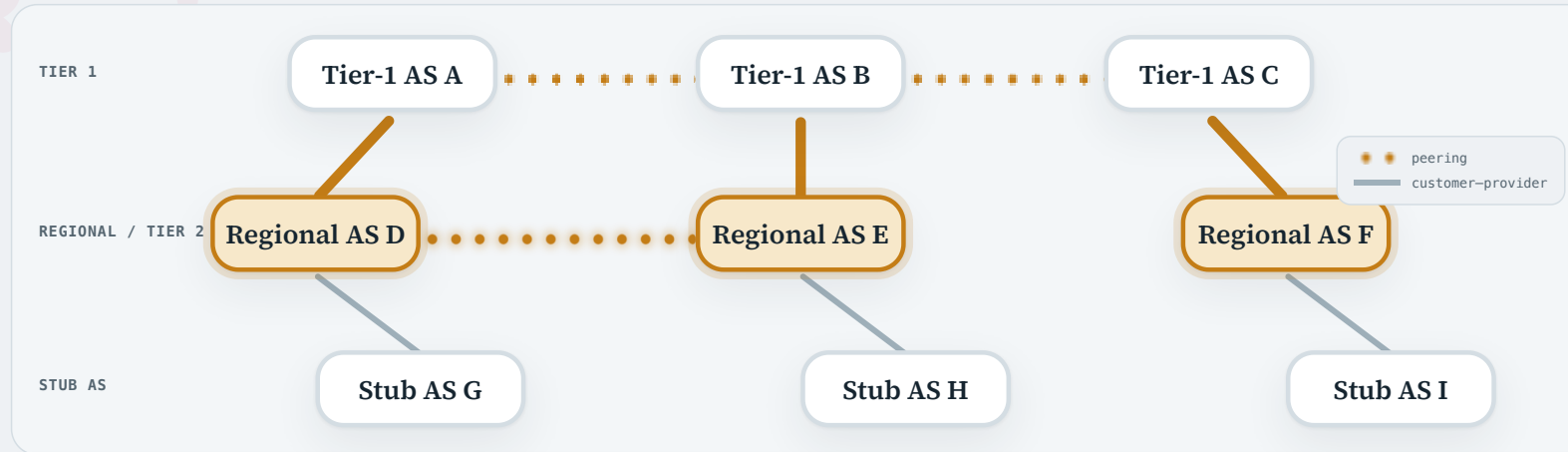
### TIER-1 TRANSIT ASEs

They commonly peer with one another and do not purchase upstream transit.

Simplified AS hierarchy and valley-free path model: CAIDA AS Relationships; Gao, 2001

HOW CUSTOMER, PROVIDER, AND PEER RELATIONSHIPS REPEAT ACROSS THE INTERNET

## These relationships create a commercial AS hierarchy



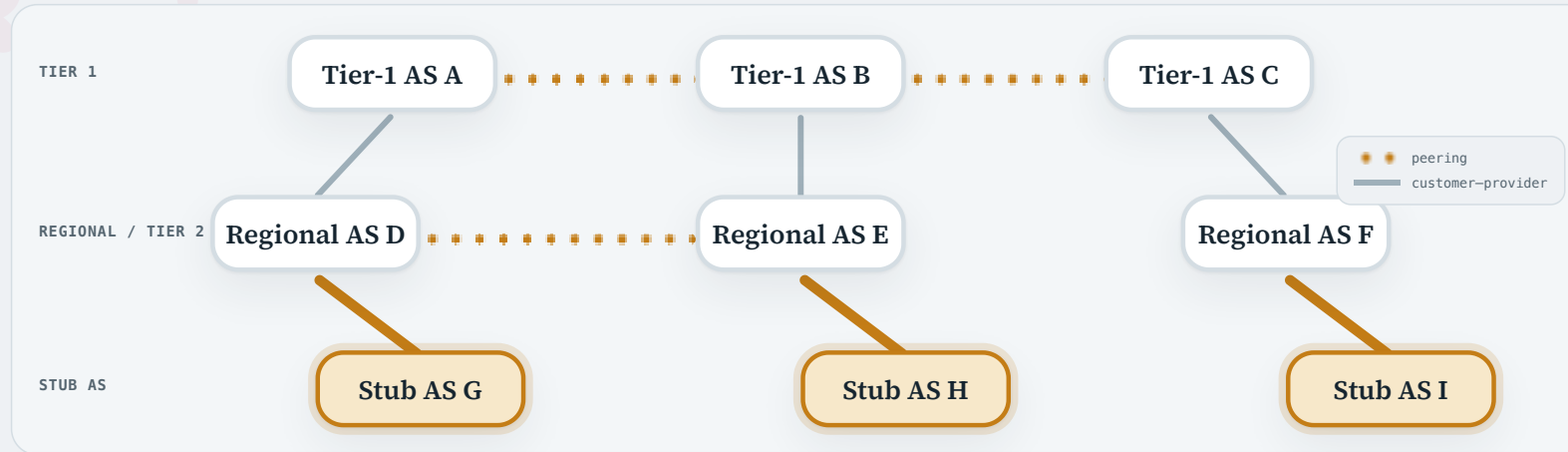
### REGIONAL TRANSIT ASes

They may buy transit upstream, sell it downstream, and peer with other regional ASes.

Simplified AS hierarchy and valley-free path model: CAIDA AS Relationships; Gao, 2001

HOW CUSTOMER, PROVIDER, AND PEER RELATIONSHIPS REPEAT ACROSS THE INTERNET

## These relationships create a commercial AS hierarchy



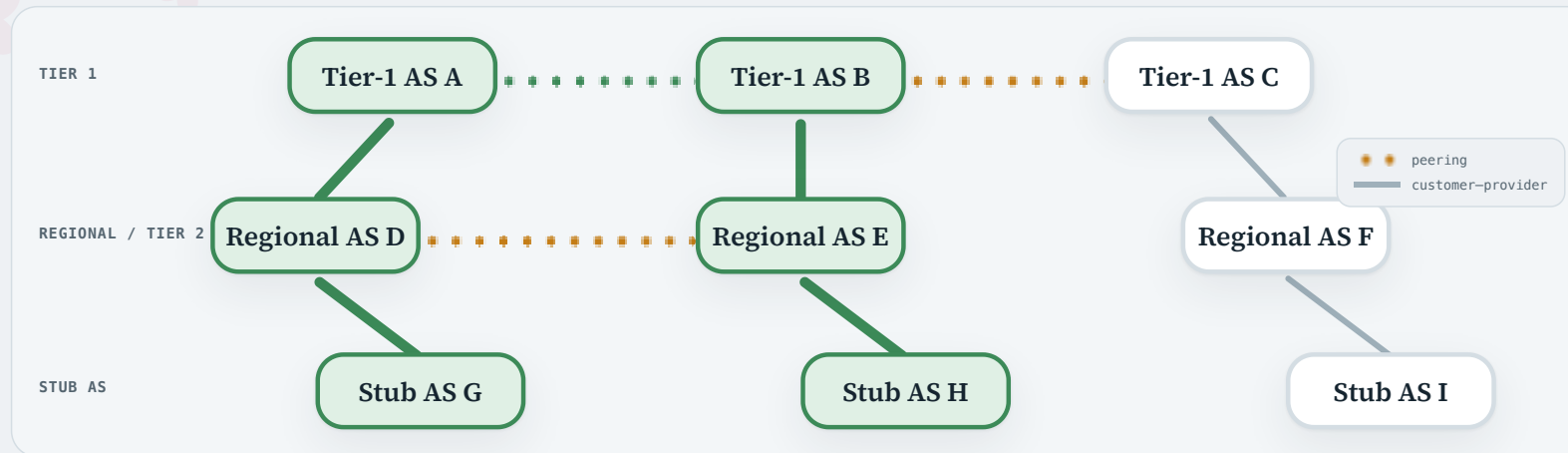
### STUB ASes

A stub reaches the wider Internet through providers and does not carry transit between other ASes.

Simplified AS hierarchy and valley-free path model: CAIDA AS Relationships; Gao, 2001

HOW CUSTOMER, PROVIDER, AND PEER RELATIONSHIPS REPEAT ACROSS THE INTERNET

## These relationships create a commercial AS hierarchy



### VALLEY-FREE PATHS

Whenever every AS's import and export policy is guided by these relationships, a path takes an **up-across-down** shape — a **valley-free path**.

e.g. a datagram from stub AS G to stub AS H climbs through providers to the Tier-1 level, crosses one peer link, then descends to AS H.

Simplified AS hierarchy and valley-free path model: CAIDA AS Relationships; Gao, 2001

# Import policy: which route should the AS prefer?

| ROUTE LEARNED FROM | COMMON PREFERENCE | ECONOMIC INTUITION                |
|--------------------|-------------------|-----------------------------------|
| Customer           | HIGH              | The customer pays us for transit. |
| Peer               | MEDIUM            | Usually settlement-free exchange. |
| Provider           | LOW               | We pay the provider for transit.  |

## 1 · CUSTOMER ROUTE

Customer traffic earns revenue, so customer-learned routes commonly receive the highest preference.

A COMMON COMMERCIAL ORDERING—NOT FIXED BGP VALUES

# Import policy: which route should the AS prefer?

| ROUTE LEARNED FROM | COMMON PREFERENCE | ECONOMIC INTUITION                |
|--------------------|-------------------|-----------------------------------|
| Customer           | HIGH              | The customer pays us for transit. |
| Peer               | MEDIUM            | Usually settlement-free exchange. |
| Provider           | LOW               | We pay the provider for transit.  |

## 2 · PEER ROUTE

A peer route is commonly preferred over a route that incurs upstream transit cost.

# Import policy: which route should the AS prefer?

| ROUTE LEARNED FROM | COMMON PREFERENCE | ECONOMIC INTUITION                |
|--------------------|-------------------|-----------------------------------|
| Customer           | HIGH              | The customer pays us for transit. |
| Peer               | MEDIUM            | Usually settlement-free exchange. |
| Provider           | LOW               | We pay the provider for transit.  |

## 3 · PROVIDER ROUTE

Provider-learned routes commonly receive low preference because the AS must pay the provider to carry that traffic.

A COMMON COMMERCIAL GUIDELINE—NOT A RULE ENFORCED BY BGP

## Export policy: where should the route be advertised?

| ROUTE LEARNED FROM | TO CUSTOMER | TO PEER | TO PROVIDER |
|--------------------|-------------|---------|-------------|
| Customer           | ✓           | ✓       | ✓           |
| Peer               | ✓           | ✗       | ✗           |
| Provider           | ✓           | ✗       | ✗           |

### TO CUSTOMERS

Customers pay for transit, so they can commonly receive every usable route.

Common relationship-based propagation guideline: RFC 7908; relationship roles: RFC 9234

A COMMON COMMERCIAL GUIDELINE—NOT A RULE ENFORCED BY BGP

## Export policy: where should the route be advertised?

| ROUTE LEARNED FROM | TO CUSTOMER | TO PEER | TO PROVIDER |
|--------------------|-------------|---------|-------------|
| Customer           | ✓           | ✓       | ✓           |
| Peer               | ✓           | ✗       | ✗           |
| Provider           | ✓           | ✗       | ✗           |

### CUSTOMER-LEARNED ROUTES

These can commonly be advertised to peers and providers because the customer pays the AS for transit.

A COMMON COMMERCIAL GUIDELINE—NOT A RULE ENFORCED BY BGP

## Export policy: where should the route be advertised?

| ROUTE LEARNED FROM | TO CUSTOMER | TO PEER | TO PROVIDER |
|--------------------|-------------|---------|-------------|
| Customer           | ✓           | ✓       | ✓           |
| Peer               | ✓           | ✗       | ✗           |
| Provider           | ✓           | ✗       | ✗           |

### PEER/PROVIDER ROUTES

Do not normally export these to another peer or provider; doing so creates unpaid transit.

# Inter-AS routing recap

- **BGP solves the inter-AS routing problem** by learning AS-level paths to destination network prefixes.
- **eBGP exchanges routes between ASes**, while **iBGP distributes candidate routes inside an AS**.
- **Both use BGP's path-vector mechanism**, carrying a destination prefix together with its AS-level path and next hop.
- **Import and export policies express AS-wide routing choices**, often driven by customer, provider, and peer relationships.

MODULE 2 • KEEPING THE INTERNET ADDRESSABLE

# IP Addressing Scaling: Aggregation, NAT and IPv6

---

1 • Intra-AS routing protocols

2 • Inter-AS routing and AS policy

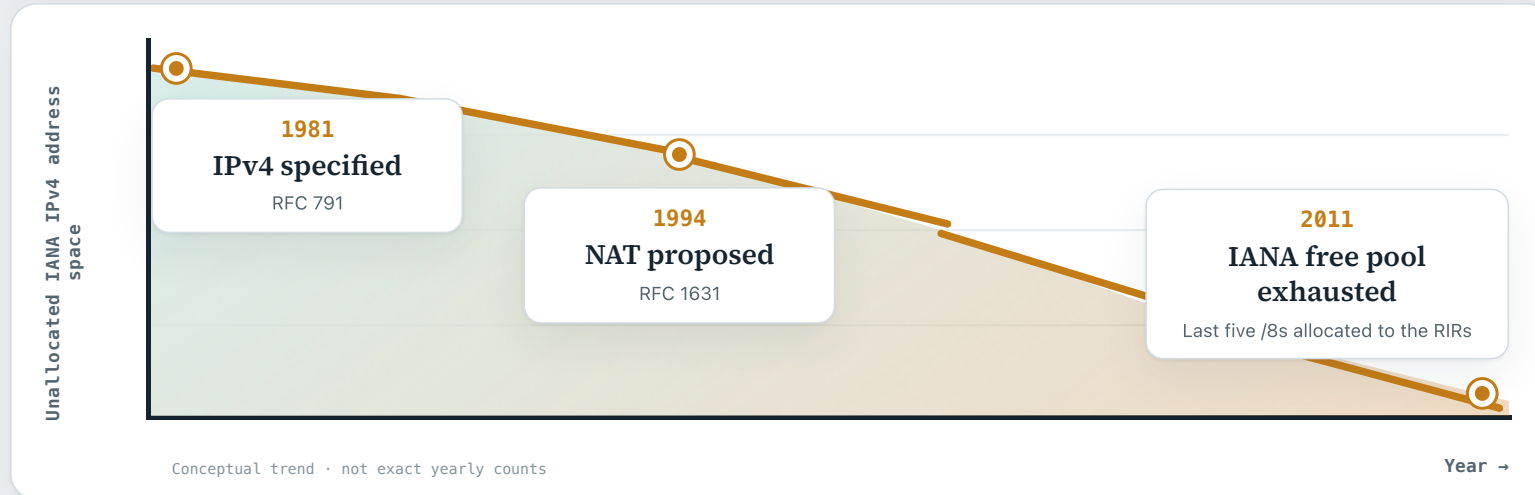
3 • Aggregation, NAT and IPv6

## A BRIEF HISTORY

# IPv4's address space became scarce as the Internet grew

**$2^{32}$  addresses**

IPv4 uses 32-bit addresses. Its finite address space became an important limitation as more networks and devices joined the Internet.



RFC Editor: RFCs 791 and 1631 · IANA central-pool exhaustion: ICANN, 3 February 2011

# How did the Internet adapt to IPv4 scarcity?

We will study three responses that extended IPv4's useful life and kept the Internet growing.

1

## Classless Inter-Domain Routing

already covered in Term 3

2

## Share public IPv4 addresses

**Network Address Translation (NAT)**

3

## Use a 128-bit address space

**IPv6**

# CIDR stopped IPv4 address space going to waste

**16 bits**

network — the split had to fall here

**16 bits**

65 534 hosts, needed or not

## THE OLD PROBLEM

- An IPv4 address is **32 bits**, and before CIDR the split between network and host could fall in only **three places** — after 8, 16 or 24 bits.
- With three sizes to choose from, a great deal of an already scarce space went unused. e.g. a site needing 500 addresses had to take a whole **16-bit** block — 65,534 host addresses, nearly all of them wasted.

# CIDR stopped IPv4 address space going to waste

**20 bits**

the university's block

**3**

department

**9 bits**

hosts inside that department

| department  | those 3 bits | its block             |
|-------------|--------------|-----------------------|
| Engineering | <b>000</b>   | <b>200.23.16.0/23</b> |
| Science     | <b>001</b>   | <b>200.23.18.0/23</b> |
| Law         | <b>010</b>   | <b>200.23.20.0/23</b> |
| Arts        | <b>011</b>   | <b>200.23.22.0/23</b> |
| ...         | <b>...</b>   | <b>...</b>            |
| Library     | <b>111</b>   | <b>200.23.30.0/23</b> |

## THE ANSWER

- Under **CIDR** the split can fall at **any bit**.
- e.g. a university takes **200.23.16.0/20** and divides it into eight sub-blocks: **3 bits** to say which department, and the remaining **9 bits** to say which host inside it.

# IPv4 address aggregation

| network     | destination    | next hop | in binary                           |
|-------------|----------------|----------|-------------------------------------|
| Engineering | 200.23.16.0/23 | uc-gw    | 11001000 00010111 00010000 00000000 |
| Science     | 200.23.18.0/23 | uc-gw    | 11001000 00010111 00010010 00000000 |
| Law         | 200.23.20.0/23 | uc-gw    | 11001000 00010111 00010100 00000000 |
| Arts        | 200.23.22.0/23 | uc-gw    | 11001000 00010111 00010110 00000000 |
| ...         | ...            | uc-gw    | ...                                 |
| Library     | 200.23.30.0/23 | uc-gw    | 11001000 00010111 00011110 00000000 |

## A NEW PROBLEM

- CIDR uses the address space far more efficiently — but it can create a problem for **router memory**.
- For example, a router carrying traffic towards the university may need to keep **all eight** of these entries, even though every one of them sends traffic to the **same next hop**.
- **How do we solve this?**

# IPv4 address aggregation

| network     | destination           | next hop | in binary                                   |
|-------------|-----------------------|----------|---------------------------------------------|
| Engineering | <b>200.23.16.0/23</b> | uc-gw    | <b>11001000 00010111 0001</b> 0000 00000000 |
| Science     | <b>200.23.18.0/23</b> | uc-gw    | <b>11001000 00010111 0001</b> 0010 00000000 |
| Law         | <b>200.23.20.0/23</b> | uc-gw    | <b>11001000 00010111 0001</b> 0100 00000000 |
| Arts        | <b>200.23.22.0/23</b> | uc-gw    | <b>11001000 00010111 0001</b> 0110 00000000 |
| ...         | ...                   | uc-gw    | ...                                         |
| Library     | <b>200.23.30.0/23</b> | uc-gw    | <b>11001000 00010111 0001</b> 1110 00000000 |

## THE ANSWER

- The answer is **address aggregation**.
- In this example, all eight entries agree on their **first 20 bits** and share a next hop. So the eight can be replaced by one entry, **200.23.16.0/20**.

# IPv4 address aggregation

| network              | destination           | next hop | in binary                                   |
|----------------------|-----------------------|----------|---------------------------------------------|
| the whole university | <b>200.23.16.0/20</b> | uc-gw    | <b>11001000 00010111 0001</b> 0000 00000000 |

## THE ANSWER

- The answer is **address aggregation**.
- In this example, all eight entries agree on their **first 20 bits** and share a next hop. So the eight can be replaced by one entry, **200.23.16.0/20**.

WHEN ONE NETWORK IS REACHED ANOTHER WAY

## What if two entries both match?

| network              | destination    | next hop | match for 200.23.18.5 |
|----------------------|----------------|----------|-----------------------|
| the whole university | 200.23.16.0/20 | uc-gw    | matches – 20 bits     |
| Science              | 200.23.18.0/23 | cloud-gw | matches – 23 bits     |

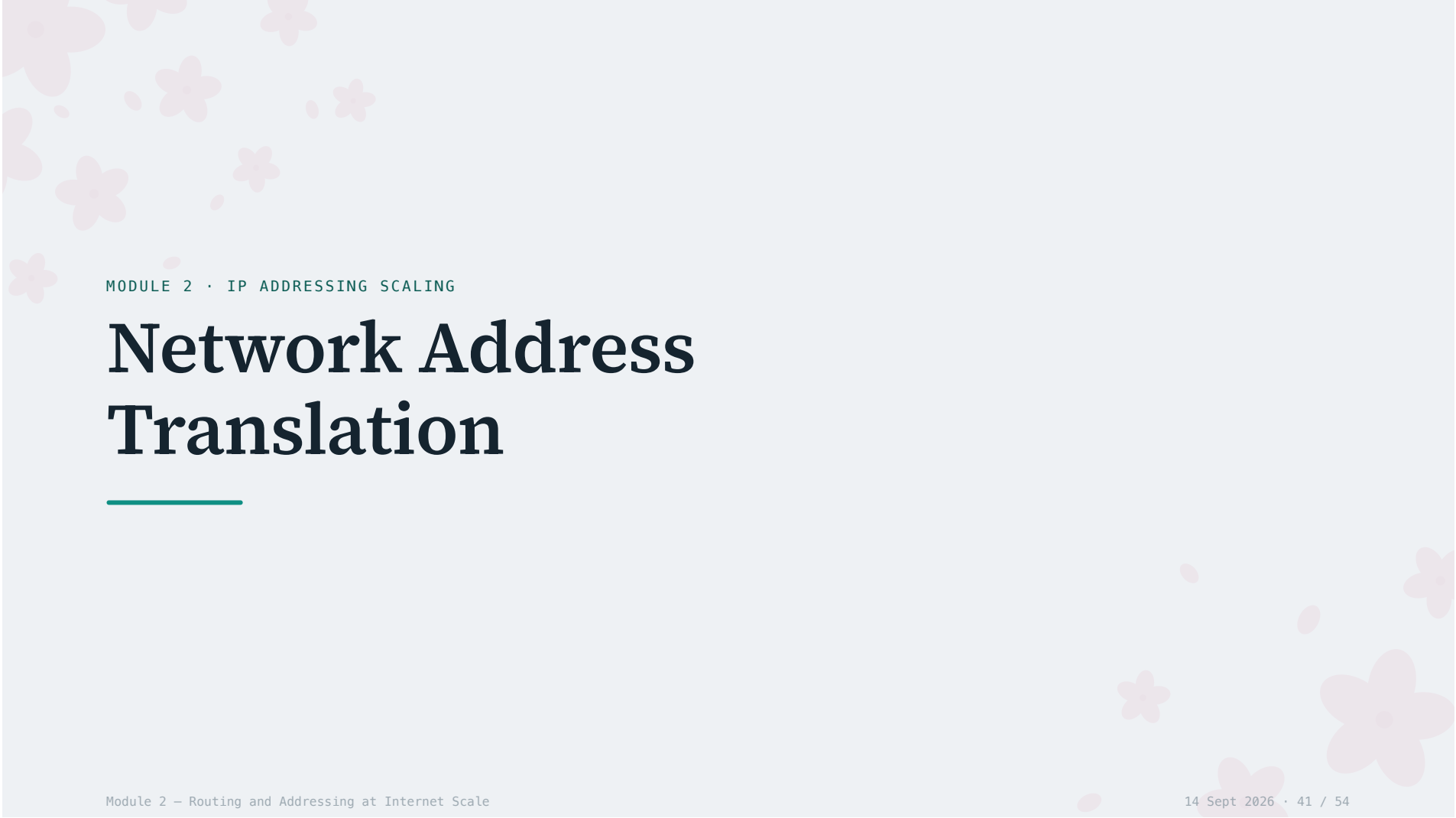
### THE FORWARDING DECISION

- The Science department is now reached through a different provider, so its **next hop is different**. A second entry, **200.23.18.0/23**, is now inserted into the router's forwarding table.
- Now suppose a datagram arrives for **200.23.18.5**. It matches **both** destination prefixes — 20 bits on one, 23 on the other.
- **Which entry should the router use?**

#### LONGEST-PREFIX MATCH

When more than one entry matches a destination address, forward using the entry with the **longest** prefix.

Here 23 beats 20, so the datagram goes to **cloud-gw**.



MODULE 2 • IP ADDRESSING SCALING

# Network Address Translation

---

# Why do so many local networks use addresses like 192.168.1.10?

- **A familiar pattern.** You may see addresses such as 192.168.1.10 on devices at home, in an office, or in a campus lab.
- **Why is there no conflict?** If both your home and a campus lab contain 192.168.1.10, why does traffic sent at home reach the home device rather than the lab device?

# NAT lets many private hosts share scarce public IPv4 addresses

The answer is **NAT** — Network Address Translation.

# NAT lets many private hosts share scarce public IPv4 addresses

Suppose a host in the home network and a host in the campus network use the **same private address**.

**Home  
network**

## Inside

A home device uses a reusable private address.

**192.168.1.10**

**Campus  
network**

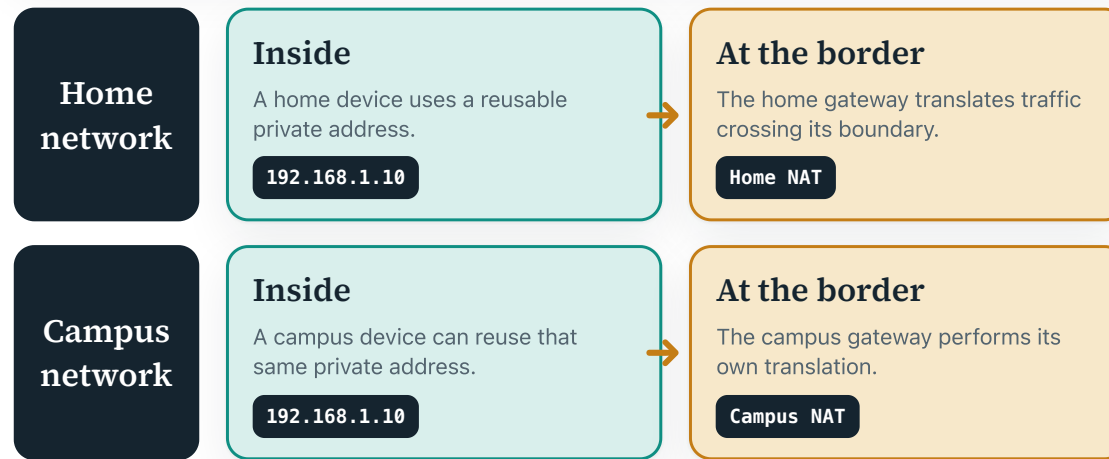
## Inside

A campus device can reuse that same private address.

**192.168.1.10**

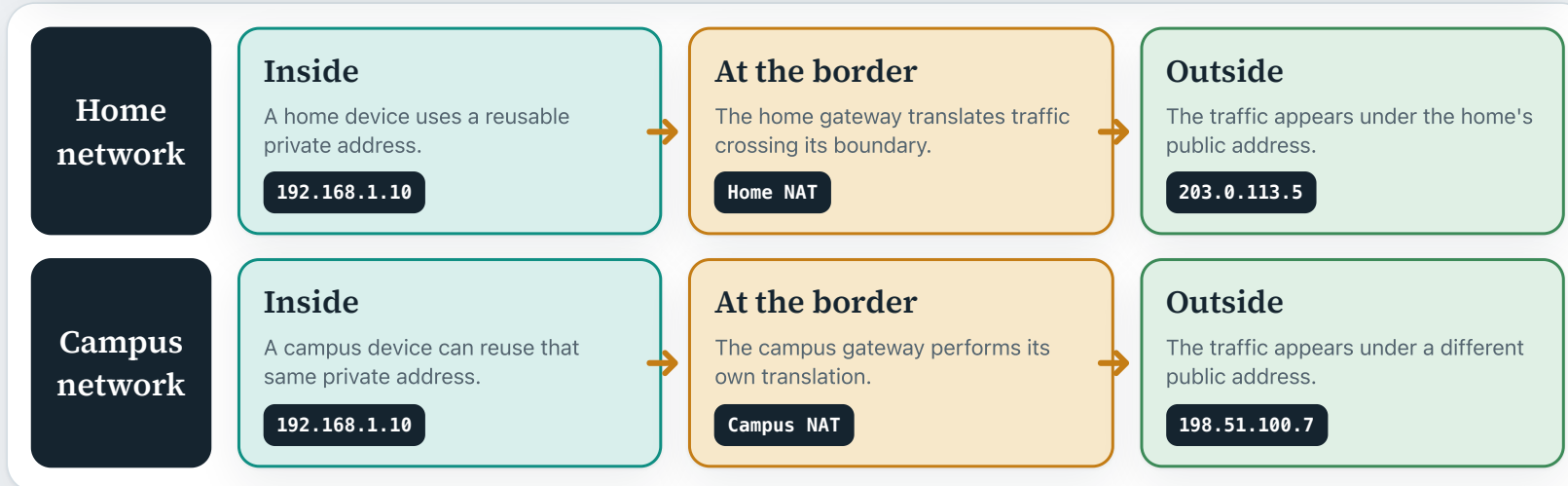
# NAT lets many private hosts share scarce public IPv4 addresses

At each network's border, NAT translates that same private address into **two different public addresses**.



# NAT lets many private hosts share scarce public IPv4 addresses

As a result, the outside world only ever observes those two public addresses — and there is **no conflict**.



# NAT translates both the address and the port

## Inside

### 1 • Private address

Suppose the home device has this private IPv4 address, assigned through **DHCP** (Dynamic Host Configuration Protocol).

**192.168.1.10**

### 2 • Source port

The device wants to send a request to the server, so it opens a **source port** for this request.

**51514**

### 3 • Original packet

From that address and port, it sends a packet to the destination server.

S 192.168.1.10:51514  
D 198.51.100.20:443

## At the border

### 4 • Public address

The home gateway has its own unique public IPv4 address, typically assigned by its ISP.

**203.0.113.5**

### 5 • Assign a public port

NAT chooses an available public port for this outbound connection and saves the mapping.

| PRIVATE SOURCE     |   | PUBLIC SOURCE      |
|--------------------|---|--------------------|
| 192.168.1.10:51514 | → | 203.0.113.5:40001  |
| [ address : port ] |   | [ address : port ] |

### 6 • Translated packet

S 203.0.113.5:40001  
D 198.51.100.20:443

## Outside

### 7 • Route to the server

The translated packet is then routed to the destination server.

**198.51.100.20:443**

# How does the reply reach the correct home device?

## Inside

### 4 · Deliver inside

The rewritten packet goes to the correct home device and port.

```
S 198.51.100.20:443
D 192.168.1.10:51514
```

## At the border

### 2 · Look up the mapping

NAT observes the public destination and finds the associated private destination.

| PUBLIC DESTINATION | PRIVATE DESTINATION  |
|--------------------|----------------------|
| 203.0.113.5:40001  | → 192.168.1.10:51514 |

### 3 · Rewrite the destination

NAT replaces the public destination with the private address and port.

```
S 198.51.100.20:443
D 192.168.1.10:51514
```

## Outside

### 1 · Reply arrives

The server replies to the public source tuple it observed.

```
S 198.51.100.20:443
D 203.0.113.5:40001
```

# Advantages and disadvantages of NAT

## Advantages

**Address conservation:** many devices can share one or a few public IPv4 addresses.

**Addressing independence:** internal addresses can change without affecting the public Internet, and the public address can change without renumbering every internal host.

## Disadvantages

**Per-flow state:** the gateway must remember active translations.

**Inbound reachability:** internal devices are not directly addressable from the public Internet. A mapping only comes into existence once the device has sent something out, so a reply can find its way back — nothing outside can start the conversation.

# IPv6

---

- IPv6 is the **newer version** of the Internet Protocol.
- It uses **128-bit addresses**, far more than IPv4's 32-bit format.

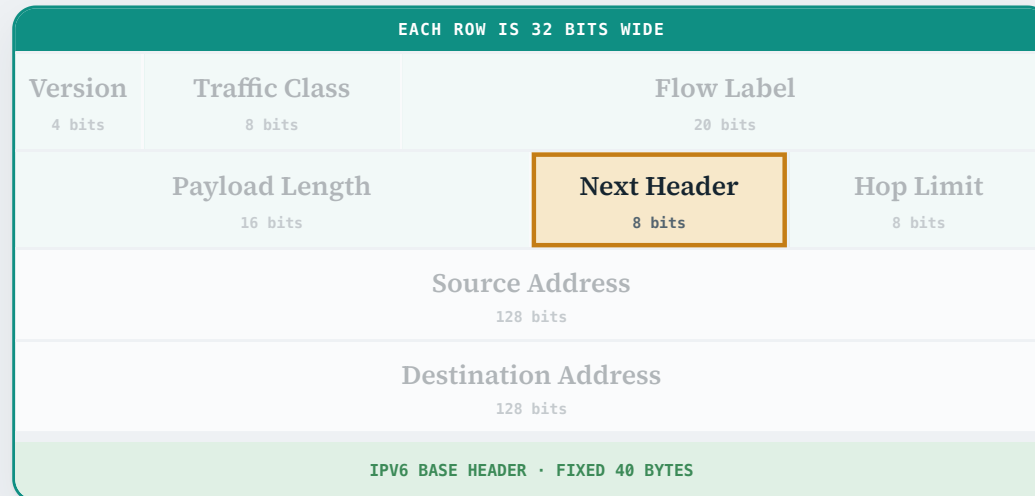
# IPv6 address

- **First 48 bits • global routing prefix:** identify an allocated site or organization block. For example, a university could be allocated this block.
- **Next 16 bits • subnet ID:** identify one subnet inside that site. For example, this could be the Engineering department's network.
- **Final 64 bits • interface identifier:** identify one interface within that subnet. For example, this could be a laptop's Wi-Fi interface.



# A breakdown of the IPv6 header

Every IPv6 packet begins with the same **fixed 40-byte base header**.



## Source and destination addresses

Both fields are 128 bits and identify the packet's endpoints.

## Hop Limit

Each router reduces it by one so a packet caught in a forwarding loop is eventually discarded.

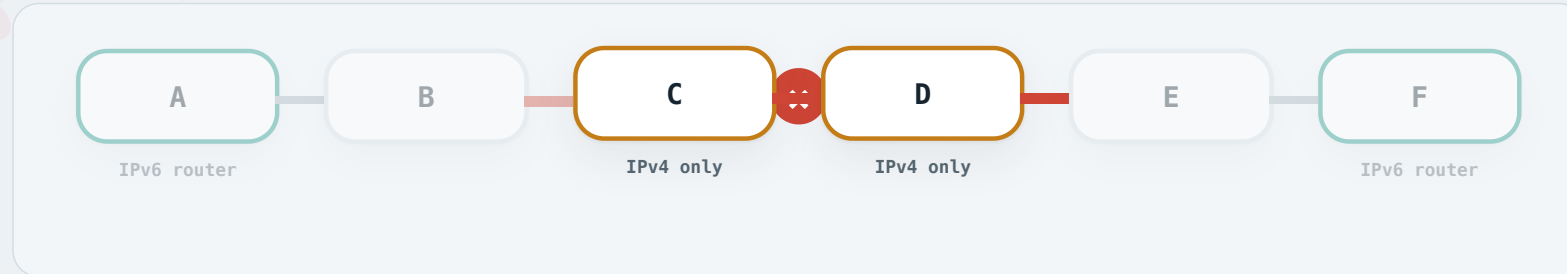
## Flow Label

Marks packets belonging to the same flow — a TCP connection, for example — so routers can treat them alike.

## Next Header

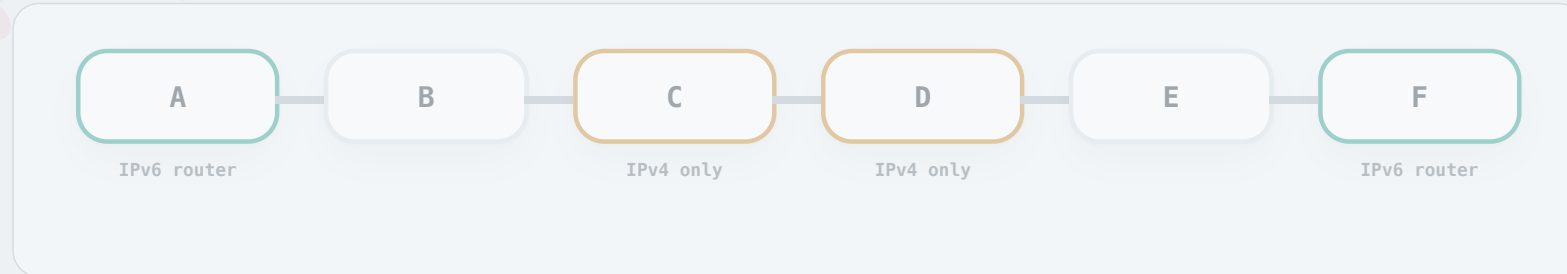
Identifies the extension header or upper-layer protocol that follows, such as a TCP or UDP header.

# IPv6 traffic may encounter IPv4-only routers



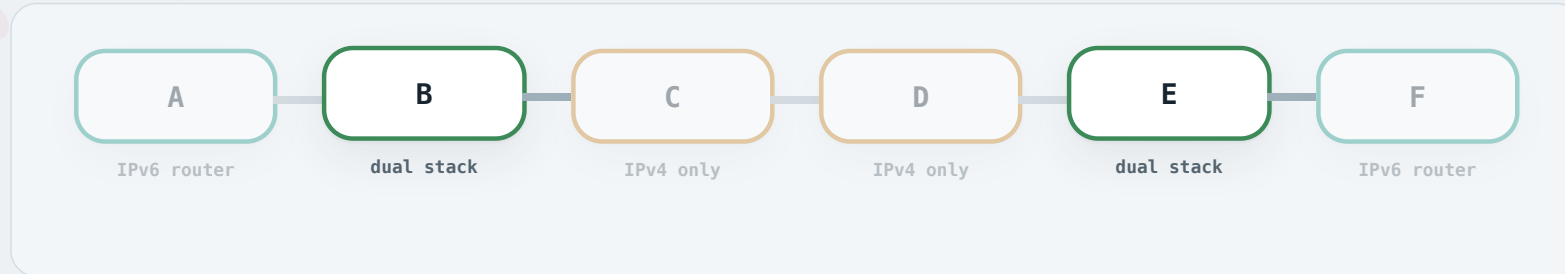
- IPv6 is still being deployed, and progress has been slow: many routers in service are older commercial devices that support only IPv4.
- Suppose IPv6 router A wants to send an IPv6 packet to IPv6 router F.
- The path crosses IPv4-only routers C and D, which cannot forward that packet directly.

# IPv6 traffic may encounter IPv4-only routers



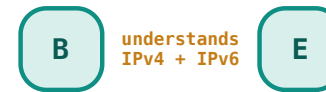
- IPv6 is still being deployed, and progress has been slow: many routers in service are older commercial devices that support only IPv4.
- Suppose IPv6 router A wants to send an IPv6 packet to IPv6 router F.
- The path crosses IPv4-only routers C and D, which cannot forward that packet directly.
- **How can we resolve this compatibility problem?**

## Two ways across the IPv4-only region

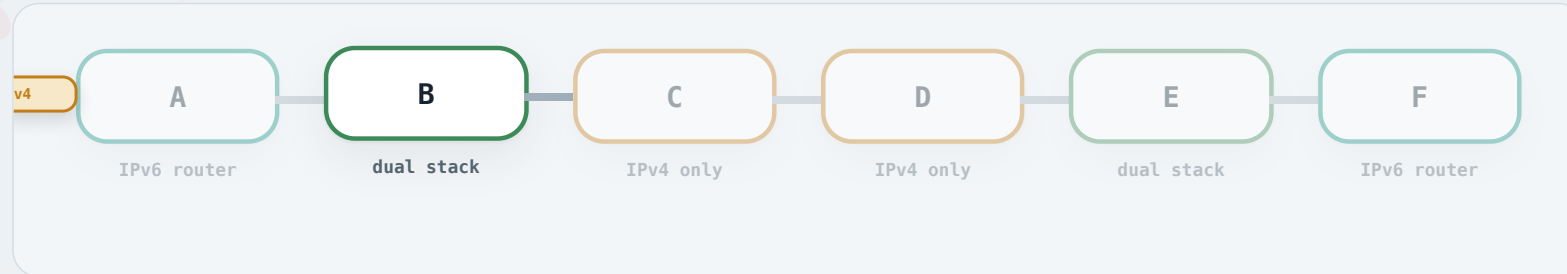


### Dual stack:

A **dual-stack router** runs both the IPv6 and the IPv4 protocol stack, so it understands either kind of packet. Here, suppose B and E are both dual-stack routers.



## Two ways across the IPv4-only region



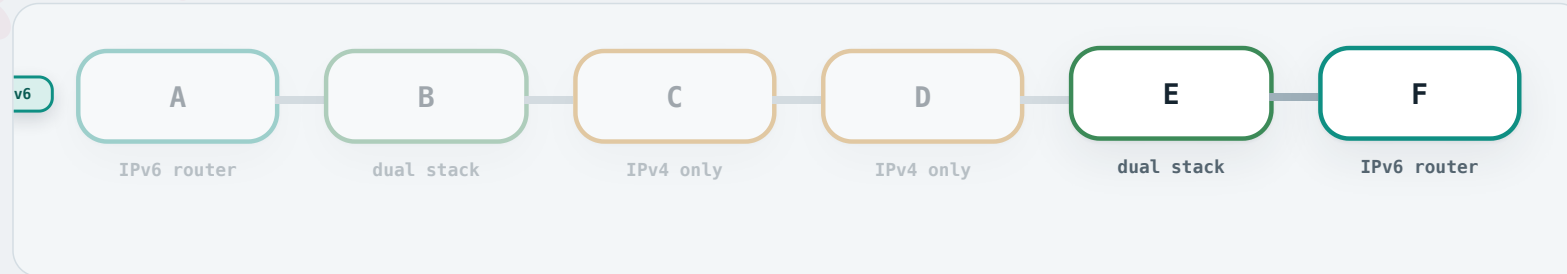
### Strategy 1 • Translation:

B rewrites the IPv6 header into an IPv4 header. What leaves B is an ordinary IPv4 packet, which C and D forward without trouble.



Fields with no IPv4 equivalent are dropped here.

## Two ways across the IPv4-only region



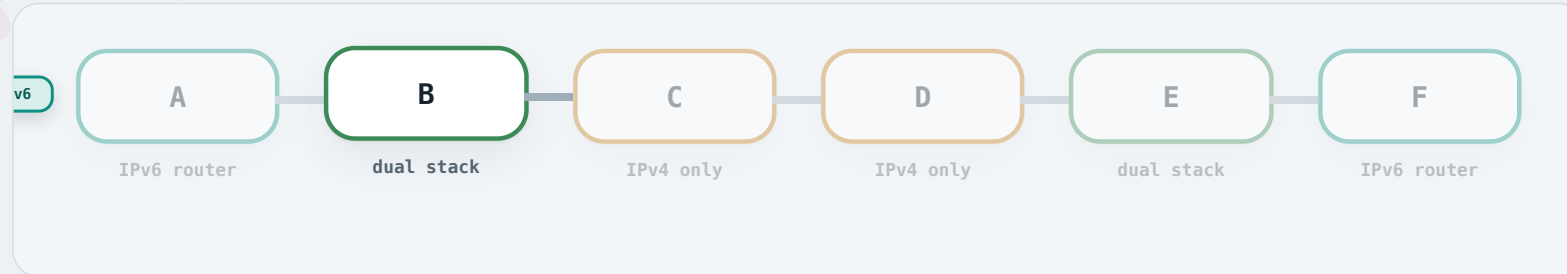
### Translate back at E:

- When the IPv4 packet reaches E, E rewrites it back into an IPv6 packet and delivers it to F.
- But IPv6 and IPv4 are not equivalent, so this translation is **lossy**: some information is dropped along the way. The IPv6 **Flow Label**, for example, has no IPv4 field to go into.



What was dropped cannot be put back.

## Two ways across the IPv4-only region



### Strategy 2 · Tunnelling:

- B **encapsulates** the entire IPv6 packet — header and payload together — into the **data field** of a new IPv4 packet.



Dual IP-layer operation and configured tunnelling: RFC 4213 §§2–3 · Stateless IP/ICMP translation: RFC 7915

## Two ways across the IPv4-only region

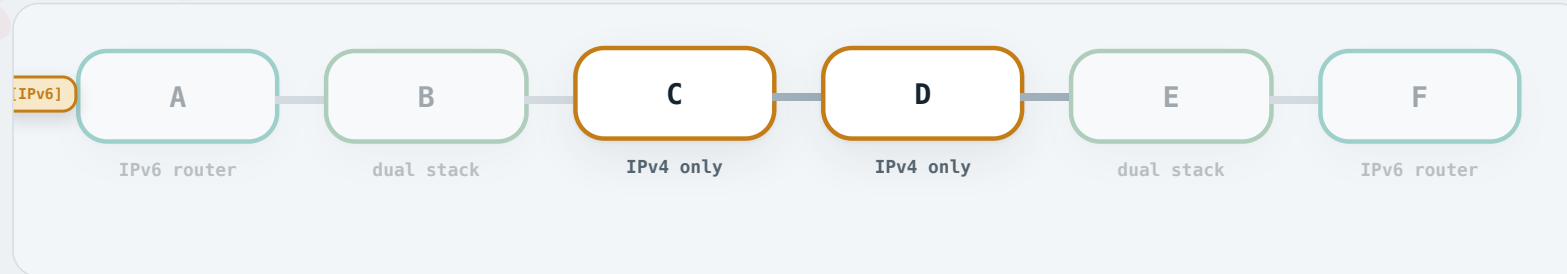


### Strategy 2 · Tunnelling:

- B **encapsulates** the entire IPv6 packet — header and payload together — into the **data field** of a new IPv4 packet.
- B then forwards that IPv4 packet to its next hop, C.



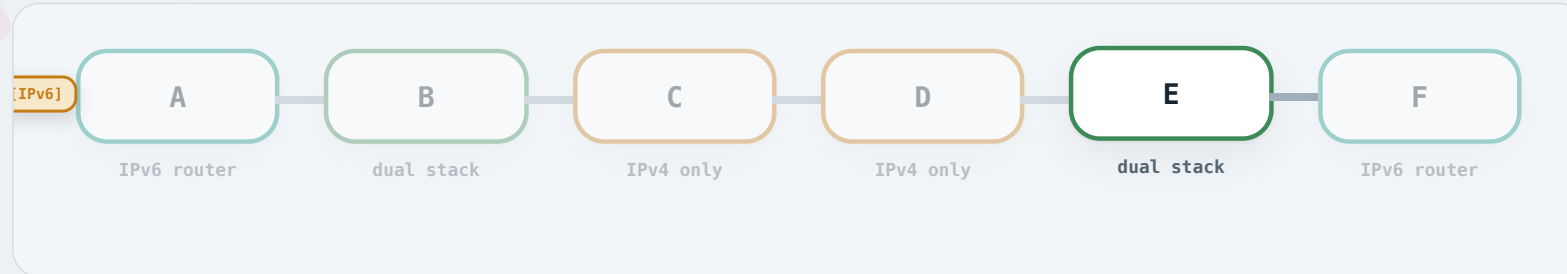
## Two ways across the IPv4-only region



### Tunnel:

The encapsulated packet crosses C and D, which see only an ordinary IPv4 packet.

## Two ways across the IPv4-only region

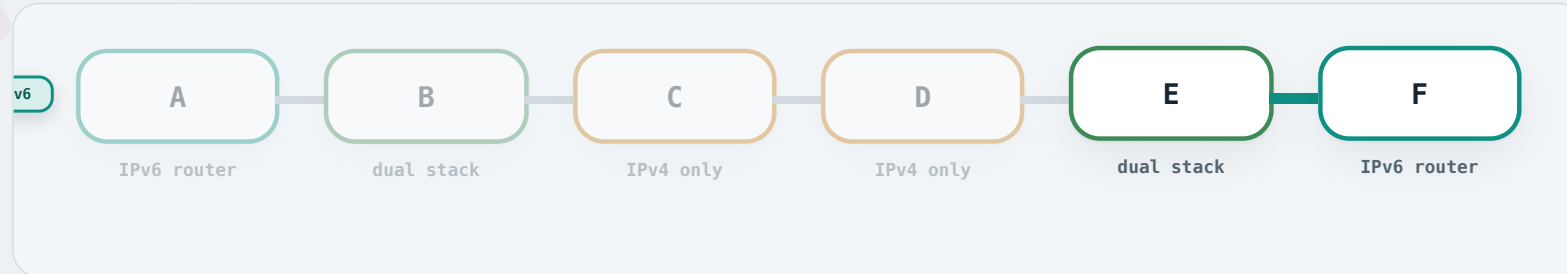


### Decapsulate at E:

E removes the outer IPv4 header, revealing the original IPv6 packet.



## Two ways across the IPv4-only region



### Delivery:

The unchanged IPv6 packet continues to F. Nothing was lost — encapsulation is **lossless**.



## Module 2 recap

- **Intra-AS routing:** RIP uses distance vector, while OSPF uses link state and areas to route within an AS.
- **Inter-AS routing:** BGP learns AS-level paths, using eBGP between ASes and iBGP within an AS.
- **IP addressing:** NAT conserves public IPv4 addresses, and IPv6 expands the address space.

# References

## William Stallings

*Data and Computer Communications*, 10th edition, Pearson, 2013 · Chapter 19, Routing.

## Andrew S. Tanenbaum & David J. Wetherall

*Computer Networks*, 5th edition, Prentice Hall, 2010 · Chapter 5, The Network Layer.

## James F. Kurose & Keith W. Ross

*Computer Networking: A Top-Down Approach Featuring the Internet*, 3rd edition, Addison Wesley, 2005 · Chapter 4, The Network Layer.

## Core Internet standards

RFC 4271, 4632, 5737, 6598, 6888, 8200, and 4213, together with the protocol-specific RFCs cited on individual frames.

## Number-resource authorities

IANA Number Resources and APNIC registry, allocation, and address-management material.

## Teaching examples

Teaching topologies, addresses, and routing policies are explicitly constructed examples unless a frame identifies an external source.

Standards and registry sources are identified on the frames where they are used.

# Acknowledgements

## **Barry Wu**

Provided the current COSC264 slides that formed the content baseline for this deck.

## **Dr Mengmeng Ge**

Developed COSC264 lecture notes used for the routing and addressing material.

## **Assoc Prof Dan Kim**

Developed earlier COSC264 lecture notes that informed the network-layer content.

## **Prof Aleksandar Kuzmanovic**

Shared CS340 lecture material used for several Internet routing examples.